



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт 7 «Робототехнические и интеллектуальные системы» Кафедра 704
Группа М7О-203М-23 Направление подготовки 09.04.01 Информатика
и вычислительная техника Магистерская программа «Обработка данных
полезных нагрузок беспилотных авиационных систем»
Квалификация магистр

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА МАГИСТРА

На тему: Решение задачи детекции и классификации дефектов на борту летательного
аппарата на основе нейросетевого подхода

Автор ВКРМ Гайдайчук Дмитрий Андреевич ()
(фамилия, имя, отчество полностью)
Научный руководитель Ляпин Никита Александрович ()
(фамилия, имя, отчество полностью)
Консультант ()
(фамилия, имя, отчество полностью)
Консультант ()
(фамилия, имя, отчество полностью)
Рецензент Лавров Александр Олегович ()

К защите допустить

Заведующий кафедрой 704 Евдокименков В.Н. ()
(№ каф) (фамилия, имя, отчество полностью)
29 март 2025 г.

Москва 2025



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт № 7 «Робототехнические и интеллектуальные системы» Кафедра 704

Группа М7О-203М-23 Направление подготовки 09.04.01 Информатика и

вычислительная техника Магистерская программа «Обработка данных

полезных нагрузок беспилотных авиационных систем»

Квалификация магистр

УТВЕРЖДАЮ

Заведующий кафедрой 704

(№ каф.)

(подпись)

Евдокименков В.Н.

(инициалы, фамилия)

2025 г.

ПЛАН РАБОТЫ

над выпускной квалификационной работой магистра

Обучающийся Гайдайчук Дмитрий Андреевич

(фамилия, имя, отчество полностью)

Руководитель Ляпин Никита Александрович

(фамилия, имя, отчество полностью)

к.т.н., доцент кафедры 704, МАИ

(ученая степень, ученое звание, должность и место работы)

1. Наименование предварительной темы (тематики) Решение задачи детекции и

классификации дефектов на борту летательного аппарата на основе нейросетевого подхода

2. Срок сдачи обучающимся законченной работы 26.05.2025

3. Цель Разработка и обучение модели сверточной нейронной сети для автоматического выявления и классификации дефектов на изображениях, полученных с камер, установленных на БПЛА, с последующей возможностью интеграции на бортовую систему для обеспечения диагностики в реальном времени.

Перечень иллюстративно-графических материалов * при наличии:

№ п/п	Наименование	Количество листов
1	Постановка цели и задач исследования	2
2	Описание и визуализация исходных данных	7
3	Методы augmentation и устойчивость моделей	4
4	Модель Mu-CNN: классификация дефектов	4
5	Модели детекции: Faster R-CNN и YOLOv11	6
6	Сравнительный анализ нейросетевых моделей	5
7	Интеграция системы и заключение	2

4. Перечень задач, решаемых для достижения поставленной цели

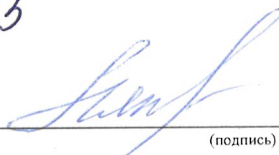
№ п/п	Наименование задачи	Срок выполнения	Примечание
1.	Анализ подходов к классификации и детекции		
2.	Сбор и предобработка изображений дефектов с БПЛА		
3.	Реализация архитектуры CNN и её обучение на тренировочном наборе данных		
4.	Валидация, тестирование и анализ точности классификации		
5.	Разработка модели детекции на базе Faster R-CNN / YOLO		
6.	Сравнительный анализ моделей и визуализация результатов		
7.	Формирование выводов, написание заключения		

5. Исходные материалы и пособия Гудфеллоу И., Бенгио Ю., Курвиль А. Глубокое обучение. – М.: ДМК Пресс, 2016., Мюллер А., Гвидо С. Введение в машинное обучение с Python. – М.: ДМК Пресс, 2016., Гомес С., Миллер Д. OpenCV. Изучаем компьютерное зрение с Python. – М.: Вильямс, 2017., Сзелиски Р. Компьютерное зрение: Алгоритмы и приложения. – М.: ДМК Пресс, 2016., Документация TensorFlow и PyTorch., Курс лекций по теме нейросетевых архитектур и компьютерного зрения (каф. 704, МАИ)., Наборы данных изображений дефектов, размеченные вручную в формате COCO., Кодовые реализации CNN, Faster R-CNN, YOLO (на базе PyTorch, TensorFlow, Ultralytics YOLOv8).

6. Дата составления плана

10.02.2025

Руководитель _____



(подпись)

Обучающийся _____



(подпись)



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

О Т З Ы В НАУЧНОГО РУКОВОДИТЕЛЯ

Обучающийся Гайдайчук Дмитрий Андреевич
Институт 7 «Робототехнические и интеллектуальные системы» Кафедра 704
Группа М7О-203М-23 Направление подготовки 09.04.01 Информатика
и вычислительная техника Магистерская программа «Обработка данных
полезных нагрузок беспилотных авиационных систем»
Квалификация магистр

Наименование темы Решение задачи детекции и классификации дефектов на борту
летательного аппарата на основе нейросетевого подхода

Руководитель Ляпин Никита Александрович, к.т.н, нет, доцент кафедры 704 МАИ
(фамилия, имя, отчество полностью, ученая степень, ученое звание, должность и место работы)

Отмеченные достоинства: Выпускная квалификационная работа студента Гайдайчука
Дмитрия Андреевича посвящена актуальной задаче автоматизированной детекции и
классификации дефектов на борту летательного аппарата с применением нейросетевых
подходов. В работе приведён обзор архитектур сверточных нейронных сетей, подробно
описаны этапы подготовки данных, построения и обучения модели. Особое внимание уделено
переходу от классической классификации к использованию детекционных моделей YOLO и
Faster R-CNN, что свидетельствует о глубоком понимании предметной области.

Студент показал высокий уровень самостоятельности, владение современными средствами
глубокого обучения и способности к критическому анализу. Работа грамотно структурирована,
хорошо оформлена и снабжена иллюстрациями. Практическая значимость подтверждается
высокой точностью модели YOLO на тестовой выборке данных.

Работа проверена на объем заимствования: заимствования — 11 %. Из них значительную часть
составляют определения и описания алгоритмов, не влияющие на оригинальность основной
части исследования.

Отмеченные недостатки: несмотря на то, что в работе подробно описаны этапы подготовки
данных и обучения модели, было бы полезно добавить более глубокий анализ альтернативных
методов детекции и классификации дефектов, таких как SSD (Single Shot MultiBox Detector) или
EfficientDet, чтобы показать, почему были выбраны именно YOLO и Faster R-CNN.

Заключение: несмотря на высказанные замечания, выпускная квалификационная работа
студента Гайдайчука Дмитрия Андреевича выполнена в полном объеме и соответствии с
заданием. Работа заслуживает высокой оценки, а ее автор — присвоения квалификации
магистр по направлению 09.04.01 «Информатика и вычислительная техника».

27 05 2025 г.

Руководитель

(подпись)



Государственный научный центр Российской Федерации
Федеральное автономное учреждение

**ГОСУДАРСТВЕННЫЙ НАУЧНО-
ИССЛЕДОВАТЕЛЬСКИЙ ИНСТИТУТ
АВИАЦИОННЫХ СИСТЕМ
(ФАУ «ГосНИИАС»)**

Юридический адрес: Викторенко ул., д.7, корп.2, г. Москва, 125319
Для почтовых отправлений: 125319, г. Москва, а/я 55
Тел.: (499) 157-70-47, факс: (499) 943-86-05, e-mail: info@gosniias.ru;
<http://www.gosniias.ru>

ОКПО: 51610303, ОГРН: 1227700109295, ИНН/КПП: 7714482225/771401001

28.05.2025 № 25-030/04

На № _____ от _____

**ЗАКЛЮЧЕНИЕ
РЕЦЕНЗЕНТА
О ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ МАГИСТРА**

Обучающийся Гайдайчук Дмитрий Андреевич

Институт 7 «Робототехнические и интеллектуальные системы» Кафедра 704

Группа М7О-203М-23 Направление подготовки 09.04.01 Информатика и
вычислительная техника Магистерская программа «Обработка данных
полезных нагрузок беспилотных авиационных систем»

Квалификация магистр

Наименование темы «Решение задачи детекции и классификации дефектов на борту
летательного аппарата на основе нейросетевого подхода»

Рецензент Лавров Александр Олегович, к.т.н., заместитель начальника подразделения по
научной работе, ФАУ «ГосНИИАС»

Отмеченные достоинства: Целью выпускной квалификационной работы студента Гайдайчука Д.А.
являлась разработка и исследование нейросетевой модели, способной решать задачу
автоматизированной детекции и классификации дефектов конструкции летательного аппарата по
изображениям, полученным с беспилотных летательных аппаратов (БПЛА). В общей части работы
представлен развернутый обзор существующих типов нейронных сетей, их архитектур, методов
обучения и способов предотвращения переобучения. Работа содержит глубокий теоретический анализ
с приведением ключевых понятий, иллюстрациями архитектур и примерами практического
применения СНС, в том числе в медицинской диагностике, видеонаблюдении и автономном
вождении. В специальной части сформулирована и подробно описана задача классификации дефектов
летательных аппаратов, приведено описание используемого набора данных, его структура, классы

повреждений и методы аугментации, направленные на повышение устойчивости модели к условиям реальной съемки. Разработана и реализована архитектура сверточной нейронной сети с использованием библиотеки TensorFlow/Keras. Также дана оценка качества модели с визуализацией результатов, а в заключении проведен анализ практической применимости модели в условиях эксплуатации. Работа характеризуется высокой степенью самостоятельности, содержит результаты, обладающие научной и практической новизной, и демонстрирует хорошее владение современными методами глубокого обучения и анализа изображений. Автор грамотно структурировал материал, корректно использовал источники и обеспечил прозрачность всех этапов эксперимента.

Отмеченные недостатки: Представленные результаты математического моделирования являются немного упрощенными и не позволяют утверждать о возможности бортовой реализации представленного алгоритма

Заключение: Несмотря на высказанные замечания, выпускная квалификационная работа студента Гайдайчука Дмитрия Андреевича выполнена в полном соответствии с заданием и предъявляемыми требованиями. Работа заслуживает высокой оценки, а ее автор — присвоения квалификации магистр по направлению 09.04.01 «Информатика и вычислительная техника».

28 05 2025 г.

Рецензент


(подпись)

Справка о проверке на наличие заимствований

Имя файла: Диплом.docx

Автор: Гайдайчук Д.А.

заглавие: Решение задачи детекции и классификации дефектов на____ борту летательного аппарата на основе нейросетевого подхода.

Год публикации: 2025

Комментарий: *Не указан*



Коллекции: Интернет 2.0, Научные статьи 2.0, Википедия, Англоязычная Википедия, Коллекция Энциклопедий, Библиотека Либрусек, Университетская библиотека, Коллекция КФУ, ВКР Российского университета кооперации, Коллекция АПУ ФСИН, Коллекция ПГУТИ, Репозиторий открытого доступа СПб гос. ун-та, Научная электронная библиотека "КиберЛенинка", ЦНМБ Сеченова, Авторефераты ВАК, Диссертации ВАК, Диссертации РГБ, Авторефераты РГБ, Готовые рефераты, ФИПС. Изобретения, ФИПС. Полезные модели, ФИПС. Промышленные образцы, Коллекция Руконт, Библиотека им. Ушинского, Готовые рефераты (часть 2), Открытые научные источники, НЭБ, БиблиоРоссика, Правовые документы I, Правовые документы II, Правовые документы III, Собрание законодательства Российской Федерации

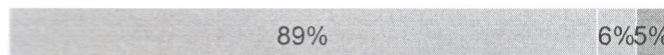
📄 Результат проверки

Оценка оригинальности документа: **89%**

Оригинальные фрагменты: 89,14%

Обнаруженные заимствования: 6,01%

Цитирование: 4,85%



Работу проверил: Ляпин Н.А.

Дата: 26.05.2025

Подпись:

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
1 ОБЩАЯ ЧАСТЬ	8
1.1 Введение в нейронные сети	8
1.1.1 Что такое нейронные сети?	8
1.1.2 Краткая история и развитие нейронных сетей	9
1.1.3 Основные типы нейронных сетей	11
1.2 Базовые принципы работы нейронных сетей	13
1.2.1 Основные компоненты: нейроны, веса, функция активации, потери ..	13
1.2.2 Процесс обучения: обратное распространение ошибок и градиентный спуск.....	14
1.3 Введение в сверточные нейронные сети (СНС).....	16
1.3.1 Что такое СНС и в чем их уникальность?	17
1.4 Архитектура сверточных нейронных сетей	20
1.4.1 Слои сверточных нейронных сетей	20
1.4.2 Примеры популярных архитектур	21
1.5 Тренировка сверточных нейронных сетей	24
1.5.1 Подготовка данных и аугментация	24
1.5.2 Проблемы переобучения и методы их решения, такие как Dropout, регуляризация	26
1.6 Примеры использования СНС в реальных приложениях	27
1.6.1 Распознавание изображений. Анализ видео	27
1.6.2 Медицинская диагностика	28
1.7 Тенденции и будущее сверточных нейронных сетей	29
1.7.1 Новые направления в архитектуре СНС	29
1.7.2. Проблемы и перспективы в развитии сверточных нейронных сетей .	31
2 СПЕЦИАЛЬНАЯ ЧАСТЬ	34
2.1 Постановка задачи классификации	34
2.2 Описание данных	37
2.2.1 Входные данные	38
2.2.2 Типы дефектов и классы	38
2.2.3 Объем данных	41

2.2.4 Алфавит классов	42
2.2.5 Реалистичность условий съемки	43
2.2.6 Предобработка данных	43
2.3 Методы аугментации данных	44
2.3.1 Повороты изображений (Rotation)	44
2.3.2 Масштабирование (Zooming)	45
2.3.3 Горизонтальное отражение (Horizontal Flip)	46
2.3.4 Изменение яркости и контрастности (Brightness and Contrast Adjustment)	47
2.3.5 Сдвиг и деформация (Shifting and Shearing)	48
2.3.6 Суммарная роль аугментации данных	48
2.4 Построение нейронной сети	49
2.4.1 Параметры модели и пути к данным	49
2.4.2 Архитектура нейронной сети	51
2.4.3 Компиляция модели	54
2.4.4 Сохранение гиперпараметров	54
2.5 Процесс обучения	55
2.5.1 Подготовка данных для обучения	56
2.5.2 Запуск обучения	58
2.5.3 Алгоритм обучения модели	59
2.5.4 Валидация модели	59
2.5.5 Оценка модели на тестовых данных	60
2.5.7 Визуализация процесса обучения	61
2.6 Результаты обучения	63
2.6.1 Графики потерь и точности	63
2.6.2 Отчет по классификации на тестовой выборке	66
2.6.3 Анализ результатов	66
2.6.4 Общая оценка производительности модели	67
2.7 Обучение моделей детекции дефектов	68
2.7.1 Методы детекции дефектов	68
2.7.2 Отчет по детекции на тестовой выборке	71
2.7.3 Общая оценка производительности моделей	72

2.8 Сценарий работы и практическое применение системы	74
2.8.1 Работа моделей в текущей реализации	74
2.8.2 Выбор модели для практического применения.....	75
2.8.3 Обработка видеопотока и вывод результатов.....	75
ЗАКЛЮЧЕНИЕ	78
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	80

ВВЕДЕНИЕ

Современная авиационная промышленность предъявляет все более высокие требования к надежности и безопасности эксплуатации летательных аппаратов. Надежность воздушных судов напрямую зависит от своевременного обнаружения и устранения дефектов конструкции, которые могут возникать как в процессе эксплуатации, так и вследствие воздействия внешних факторов. Особенно остро стоит задача мониторинга состояния элементов обшивки, соединительных узлов и других конструктивно значимых компонентов, поскольку даже незначительные на первый взгляд повреждения могут привести к серьезным аварийным ситуациям.

Традиционные методы диагностики, в частности визуальный осмотр и неразрушающий контроль с участием специалистов, продолжают широко применяться на практике. Однако эти методы обладают рядом ограничений: высокая трудоемкость, зависимость от квалификации персонала, сложность доступа к определенным участкам конструкции и невозможность автоматической обработки большого количества информации. Кроме того, подобные подходы не всегда обеспечивают оперативность и полноту анализа, особенно при осмотре крупногабаритных объектов в условиях ограниченного времени, например, при техническом обслуживании между полетами.

В последние годы активное развитие технологий искусственного интеллекта и компьютерного зрения открыло новые возможности для автоматизации процессов диагностики. Одним из наиболее перспективных направлений является применение нейросетевых моделей, в частности сверточных нейронных сетей (СНС), для анализа изображений с целью выявления и классификации дефектов. Эти модели способны обрабатывать визуальную информацию с высокой точностью, извлекать сложные признаки и устойчиво работать в условиях реальных помех, таких как изменяющееся освещение, ракурсы съемки и наличие посторонних объектов на изображении.

Использование беспилотных летательных аппаратов (БПЛА) в качестве платформы для сбора визуальной информации дополнительно усиливает эффективность такой системы. БПЛА позволяют производить осмотр воздушных судов с разных ракурсов, включая труднодоступные участки, и выполнять съемку с высокой детализацией. В сочетании с алгоритмами глубокого обучения это обеспечивает возможность создания полностью автоматизированных решений для мониторинга технического состояния авиационной техники.

Целью данной работы является разработка и исследование нейросетевой модели, способной решать задачу детекции и классификации дефектов конструкции летательного аппарата по изображениям, полученным с БПЛА. Особое внимание уделяется устойчивости модели к внешним условиям, реалистичности применяемых данных и практической применимости предложенного подхода.

В ходе выполнения работы была проведена комплексная подготовка обучающих данных, включающая аугментацию, нормализацию и категоризацию изображений. Далее была спроектирована и реализована архитектура сверточной нейронной сети, оптимизированная под специфику задачи. Также произведено обучение модели и оценка ее производительности на специально отобранных тестовых данных.

Научная новизна и практическая значимость данного исследования заключаются в применении современных методов глубокого обучения к задачам авиационного контроля, что позволяет повысить точность, оперативность и автономность диагностики конструктивных повреждений. Результаты могут быть использованы при построении интеллектуальных систем технического обслуживания в гражданской и военной авиации, а также в других отраслях, где необходим автоматизированный контроль состояния крупногабаритных технических объектов.

1 ОБЩАЯ ЧАСТЬ

1.1 Введение в нейронные сети

1.1.1 Что такое нейронные сети?

Нейронные сети — это алгоритмы машинного обучения, вдохновленные структурой и функционированием мозга. Основным строительным блоком нейронной сети является искусственный нейрон, который имитирует работу биологического нейрона. Нейроны в искусственных нейронных сетях обрабатывают входные данные и передают сигналы другим нейронам, что позволяет сети выполнять различные задачи, такие как распознавание образов, обработка естественного языка и даже игра в игры на уровне или лучше, чем человек. [1]

История и вдохновение. Концепция нейронных сетей возникла из желания воспроизвести когнитивные способности человеческого мозга в машине. Уже в 1943 году ученые Уоррен Маккаллок и Уолтер Питтс представили модель нейрона, которая стала основой для развития искусственных нейронных сетей. Они описали, как нейроны могут использовать логические операции для обработки информации. [2]

Структура нейронных сетей. Искусственные нейронные сети состоят из трех основных слоев: входного слоя, одного или нескольких скрытых слоев и выходного слоя. На рисунке 1.1 представлена типичная архитектура многослойной нейронной сети, где каждый нейрон соединен с нейронами следующего слоя через обучаемые веса. Входной слой принимает сырые данные, скрытые слои обрабатывают данные через взвешенные соединения, а выходной слой предоставляет результат. Соединения между нейронами называются весами, и именно их корректировка в процессе обучения позволяет сети учиться и адаптироваться к задачам. [3]

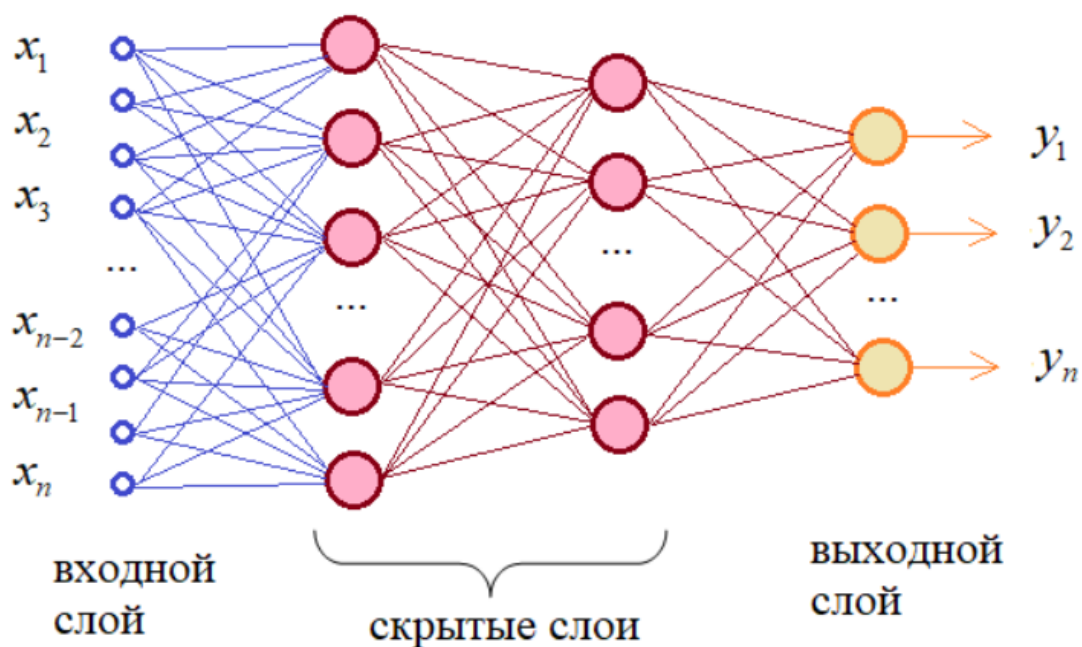


Рисунок 1.1 – Архитектура нейронной сети.

Принципы обучения. Обучение нейронной сети включает в себя корректировку весов на основе ошибок, которые сеть делает в прогнозах. Этот процесс называется обратным распространением ошибок, где ошибка между предсказанным и истинным значениями распространяется обратно по сети, позволяя ей учиться и оптимизировать свои веса. [4]

Примеры использования. Нейронные сети находят применение в самых разных областях, начиная от рекомендательных систем и заканчивая автономными автомобилями. Они могут распознавать и интерпретировать человеческую речь, улучшать медицинскую диагностику и даже управлять сложными системами в реальном времени. [5]

Это только начало обширной темы нейронных сетей. Далее мы более подробно рассмотрим разные типы нейронных сетей, включая сверточные нейронные сети, и их уникальные особенности и применения.

1.1.2 Краткая история и развитие нейронных сетей

История нейронных сетей начинается с попыток моделирования процессов, происходящих в человеческом мозге, и развивается в важную отрасль искусственного интеллекта с многочисленными приложениями.

Ранние годы (1940-е — 1960-е).

1943 год: Уоррен Маккаллок и Уолтер Питтс опубликовали работу, представляющую нейронные сети как наборы бинарных выходных элементов, которые могут воспроизводить простейшие логические функции. [2]

1958 год: Фрэнк Розенблатт создал Перцептрон, первую учебную модель для нейронных сетей, которая способна обучаться и делать простейшие прогнозы, основанные на весах входных данных. [6]

Затишье и период зимы ИИ (1970-е — начало 1980-х).

После первоначального восторга, интерес к нейронным сетям упал из-за ограничений ранних моделей, таких как неспособность Перцептрона решать задачи XOR, что было подчеркнуто в книге Марвина Мински и Сеймура Пейперта «Перцептроны» (1969). Это привело к снижению финансирования и исследований в области нейронных сетей.

Возрождение интереса (1980-е — 1990-е).

1982 год: Джон Хопфилд представил сеть Хопфилда, которая могла служить формой ассоциативной памяти. [7]

1986 год: Дэвид Румельхарт, Джеффри Хинтон и Рональд Уильямс ввели алгоритм обратного распространения ошибок, который значительно улучшил обучение многослойных перцептронов. [4]

1990-е: Прорыв в разработке алгоритмов и улучшенная вычислительная мощность привели к возрождению интереса и развитию нейронных сетей. Это также включало введение Лонг-Шорт Терм Мемори (LSTM) сетей для работы с последовательными данными. [8]

Современное развитие (2000-е — настоящее время).

2006 год: Джеффри Хинтон ввел понятие "глубоких верований сетей", что способствовало развитию глубокого обучения.

2012 год: Алекс Крижевский и его команда из Торонто использовали глубокую сверточную нейронную сеть (AlexNet) для выигрыша соревнования ImageNet, значительно улучшив результаты распознавания изображений. [9]

С тех пор нейронные сети стали основой множества продуктов и технологий, используемых в повседневной жизни, от автоматического перевода и анализа изображений до автономного вождения.

Эта история подчеркивает важность нейронных сетей как основного направления в исследованиях искусственного интеллекта. Они продолжают развиваться и находят все новые применения, благодаря улучшению алгоритмов, росту вычислительных мощностей и расширению наборов данных для обучения. Нейронные сети оказали значительное влияние на многие аспекты технологий и повседневной жизни, от повышения точности медицинских диагнозов до улучшения способов взаимодействия человека с машинами. Эволюция нейронных сетей подчеркивает потенциал искусственного интеллекта как ключевого драйвера будущих инноваций.

1.1.3 Основные типы нейронных сетей

Нейронные сети делятся на несколько типов в зависимости от их архитектуры и предназначения. Рассмотрим основные из них:

1) Перцептроны.

Однослойные перцептроны: это самая простая форма нейронной сети, содержащая только один слой нейронов, которые напрямую соединяют входные данные с выходом. Их возможности ограничены, так как они могут обучаться только разделять данные линейно. [6]

Многослойные перцептроны (MLP): Эти сети содержат один или несколько скрытых слоев между входным и выходным, что позволяет им решать более сложные задачи, которые не могут быть решены линейными методами. [4]

2) Сверточные нейронные сети (CNN).

Сверточные нейронные сети (CNN) особенно хорошо подходят для анализа визуальной информации и часто используются в задачах компьютерного зрения, таких как распознавание изображений и видеоанализ. CNN используют свертки вместо общих матричных умножений в полносвязных слоях, что позволяет им более эффективно работать с

изображениями (см. рисунок 1.2) и видеоанализ. CNN используют свертки вместо общих матричных умножений в полносвязных слоях, что позволяет им более эффективно работать с изображениями. [3]

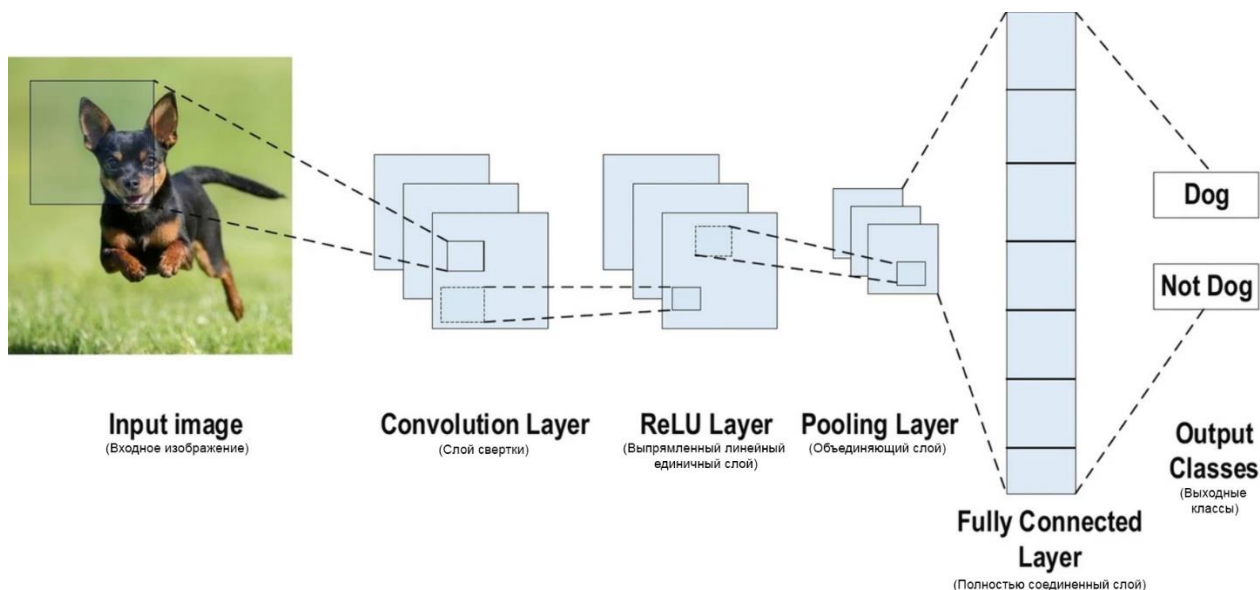


Рисунок 1.2 – Этапы обработки изображения в сверточной нейронной сети: от входного изображения до выхода через сверточный, ReLU, объединяющий и полносвязный слой.

3) Рекуррентные нейронные сети (RNN).

RNN способны обрабатывать последовательные данные (например, текст или временные ряды), поскольку они могут передавать информацию через последовательности. Это делает их подходящими для таких задач, как машинный перевод, распознавание речи и предсказание временных рядов. [10]

4) Сети с долгой краткосрочной памятью (LSTM).

Вариант RNN, который решает проблему исчезающего градиента, позволяя информации сохраняться в сети на более длительный срок. Это делает LSTM особенно полезным в задачах, где необходимо учитывать контекст на больших временных промежутках. [8]

5) Генеративно-состязательные сети (GAN).

GAN состоят из двух сетей, соревнующихся друг с другом: генератора, который создает данные, и дискриминатора, который пытается отличить поддельные данные от настоящих. Эти сети нашли широкое применение в генерации изображений, видео и даже музыки. [11]

6) Сети преобразования внимания (Transformers).

Основанные на механизмах внимания, трансформеры улучшили способность моделей к обработке последовательностей, особенно в задачах обработки естественного языка. Трансформеры стоят за такими инновациями, как GPT (для генерации текста) и BERT (для понимания текста). [12]

Каждый из этих типов нейронных сетей имеет уникальные архитектурные особенности и предназначен для решения определенных типов задач. Это лишь обзор основных типов, и существует множество вариаций и специализаций в каждой категории, адаптированных под конкретные приложения и требования.

1.2 Базовые принципы работы нейронных сетей

Нейронные сети, будучи основой современных систем машинного обучения, строятся на нескольких ключевых компонентах: нейронах, весах, функциях активации и функциях потерь. Понимание этих элементов критически важно для освоения работы нейронных сетей.

1.2.1 Основные компоненты: нейроны, веса, функция активации, потери

Нейроны — это основные вычислительные единицы нейронной сети. Каждый нейрон получает входные данные, производит их обработку и передает результат дальше по сети. В биологическом аналоге, это клетки, которые передают нервные сигналы. В искусственных нейронных сетях нейрон моделируется как сумматор входных сигналов, взвешенных определенными коэффициентами, с последующей обработкой суммы через функцию активации. [13]

Веса — это параметры в нейронной сети, которые настраиваются в процессе обучения. Они определяют важность каждого входного сигнала для нейрона. В начале обучения веса обычно инициализируются случайным образом, а затем корректируются для минимизации ошибки в прогнозах сети. Механизм обновления весов через обратное распространение и градиентный спуск является ключевым в обучении нейронных сетей. [6]

Функция активации в нейроне определяет, будет ли и как сильно нейрон активироваться в ответ на полученные входные данные. Функция активации вносит нелинейность в процесс обработки данных, что позволяет нейронным сетям учиться и моделировать сложные закономерности и отношения в данных. Распространенные примеры функций активации включают сигмоидную функцию, гиперболический тангенс, ReLU (Rectified Linear Unit) и другие. [4]

Функция потерь измеряет, насколько предсказания нейронной сети отличаются от фактических значений. Она является критерием, по которому определяется качество модели, и ее минимизация является основной задачей в процессе обучения. Существуют различные типы функций потерь, подходящие для разных типов задач, такие как среднеквадратичная ошибка для регрессии и кросс-энтропия для классификации. [14]

Эти компоненты взаимодействуют в процессе обучения и работы нейронной сети, обеспечивая ее способность адаптироваться и учиться решать различные задачи. Нейронные сети могут варьироваться по архитектуре и сложности, но основные принципы их работы остаются постоянными и определяют их универсальность и мощь в машинном обучении.

1.2.2 Процесс обучения: обратное распространение ошибок и градиентный спуск

Обучение нейронных сетей основывается на двух ключевых концепциях: обратном распространении ошибок и градиентном спуске. Эти

методы позволяют сети адаптироваться и улучшаться, минимизируя ошибку в прогнозах посредством корректировки весов.

Обратное распространение ошибок. Обратное распространение ошибок — это алгоритм, используемый для эффективного вычисления градиента функции потерь по отношению к каждому весу в нейронной сети. Это делается для того, чтобы определить, как каждый вес влияет на ошибку, и соответственно насколько и в каком направлении его нужно изменить, чтобы уменьшить ошибку. Процесс обратного распространения включает несколько шагов: [8]

Прямое распространение: во время прямого распространения данные передаются от входного слоя к выходному, через один или несколько скрытых слоев, и для каждого входа вычисляется выход сети.

Вычисление ошибки: Ошибка (или потеря) вычисляется с использованием функции потерь, которая сравнивает предсказанные выходные данные сети с истинными значениями.

Обратное распространение: затем ошибка распространяется обратно от выходного слоя к входному. На каждом этапе вычисляется градиент функции потерь по отношению к каждому весу, что показывает, как нужно изменить веса, чтобы минимизировать ошибку.

Градиентный спуск. Градиентный спуск — это метод оптимизации, используемый для минимизации функции потерь путем итеративного изменения параметров модели (весов) в направлении, противоположном градиенту функции потерь (рисунок 1.3). Существует несколько вариантов градиентного спуска:

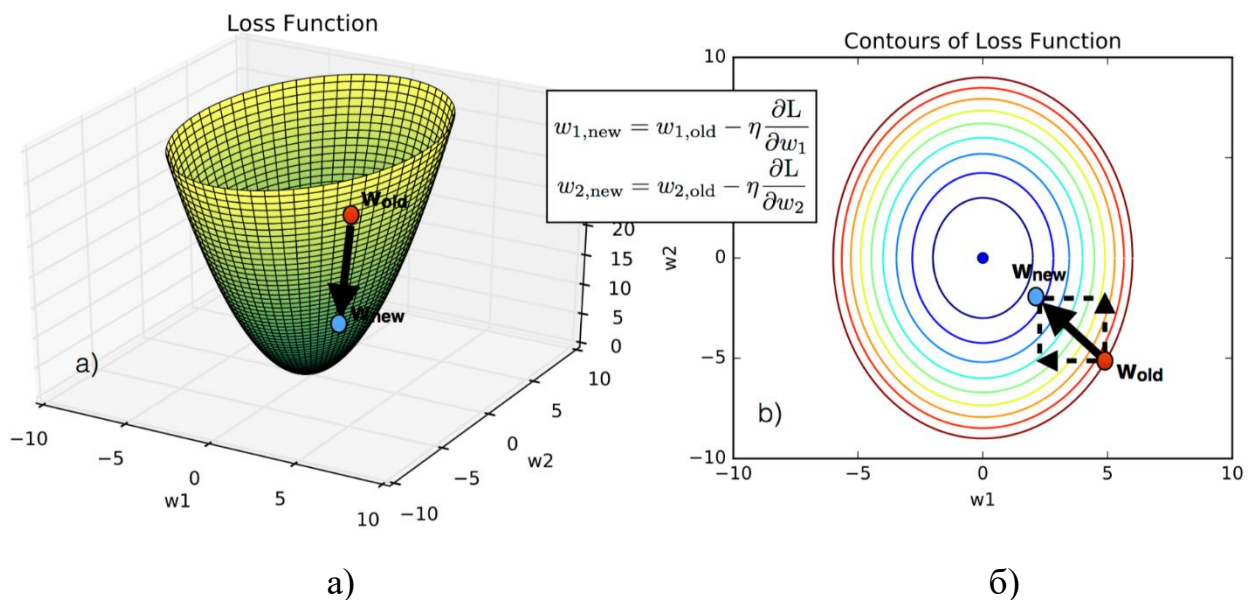


Рисунок 1.3 – Визуализация алгоритма градиентного спуска:

- (а) — поверхность функции потерь с направлением движения к минимуму;
- (б) — контурные линии функции потерь с указанием направления обновления весов.

Пакетный градиентный спуск: Весь набор данных используется для вычисления градиента и обновления весов за один шаг. [15]

Стохастический градиентный спуск (SGD): Веса обновляются для каждого обучающего примера. Это может привести к более быстрой сходимости, но с большей вариативностью в обновлениях весов. [16]

Мини-пакетный градиентный спуск: Компромисс между двумя предыдущими методами; набор данных делится на маленькие пакеты (батчи), и веса обновляются после обработки каждого пакета. [17]

Эти методы позволяют нейронной сети постепенно улучшать свои предсказания, минимизируя ошибку в прогнозах и улучшая общую точность модели. Обратное распространение и градиентный спуск вместе образуют основу для обучения большинства типов нейронных сетей, позволяя им адаптироваться к сложным задачам и большим объемам данных.

1.3 Введение в сверточные нейронные сети (СНС)

Сверточные нейронные сети (СНС) представляют собой один из типов глубоких нейронных сетей, который специально разработан для анализа

данных, имеющих вид решетки, таких как изображения. В отличие от полносвязных нейронных сетей, структура которых представлена на рисунке 1.4, СНС способны эффективно извлекать пространственные признаки из визуальной информации. [4]

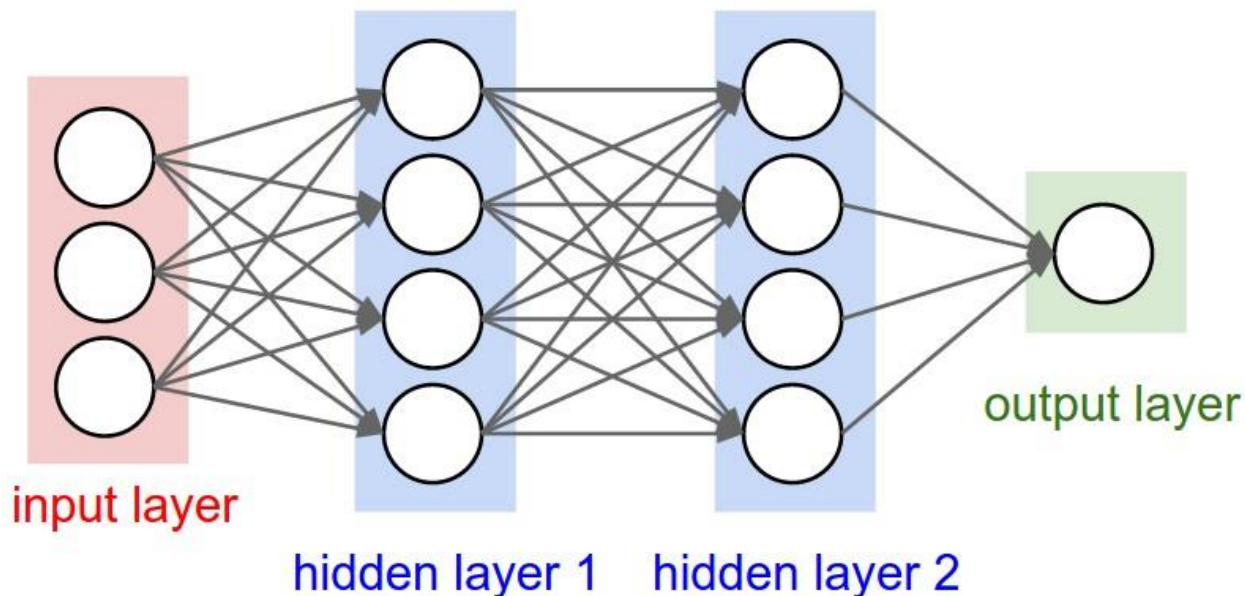


Рисунок 1.4 – Архитектура стандартной полносвязной нейронной сети с двумя скрытыми слоями: входной слой, два скрытых слоя и выходной слой.

1.3.1 Что такое СНС и в чем их уникальность?

Особенности сверточных нейронных сетей:

1) Локальное восприятие: В отличие от полносвязных слоев, где каждый нейрон обрабатывает информацию от всех входных нейронов, сверточные слои фокусируются на локальных областях входных данных. Это позволяет им извлекать основные признаки из маленьких пространственных областей данных, что очень эффективно для обработки изображений. [9]

2) Свертка: Основной элемент СНС — сверточный слой, который применяет фильтры к исходному изображению для создания карты признаков. Фильтры, или ядра, перемещаются по изображению, и на каждом шаге производятся операции свертки, которые фактически являются взвешенными суммами в рамках локального региона изображения. [1]

3) Инвариантность к положению: Одна из ключевых особенностей СНС — способность обнаруживать объекты независимо от их положения на

изображении. Когда сеть учится распознавать определенный объект, она сможет распознавать его в любом месте, что является результатом свойства сверточных сетей делать обучаемые фильтры инвариантными к смещению. [18]

4) Пулинг (subsampling или downsampling): после нескольких сверточных слоев обычно следует слой пулинга, который уменьшает размерность карт признаков, уменьшая количество параметров и вычислений в сети. Это помогает уменьшить переобучение, сохраняя при этом важные признаки. [19]

5) Глубина сети: СНС обычно содержат много слоев, где начальные слои извлекают простые признаки, такие как края или цвета, а последующие слои комбинируют их для формирования более сложных образов, таких как части объектов или целые объекты (см. рисунок 1.5). [20]

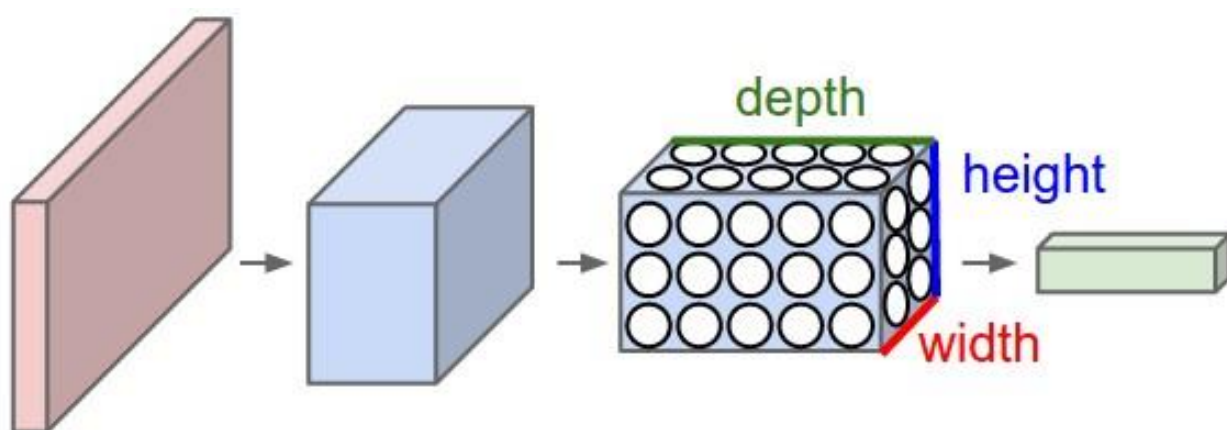


Рисунок 1.5 – Обработка изображения в сверточной нейронной сети: наглядное представление формирования объемных карт признаков с тремя пространственными измерениями — шириной, высотой и глубиной.

Уникальность СНС заключается в их способности автоматически и эффективно извлекать важные признаки из больших и сложных наборов визуальных данных без необходимости ручной предварительной обработки данных. Это делает СНС идеальными для задач компьютерного зрения, таких как автоматическое распознавание изображений и видео, медицинская диагностика изображений, и даже в области обработки естественного языка, где они используются для анализа структуры текста.

1.3.2 Применение СНС в задачах компьютерного зрения

Сверточные нейронные сети (СНС) нашли широкое применение в различных областях компьютерного зрения благодаря их высокой эффективности в обработке и анализе изображений. Вот несколько ключевых задач, где СНС оказались особенно полезными:

1) Распознавание изображений СНС могут классифицировать изображения в predetermined категории с высокой точностью. Это стало возможным благодаря способности СНС извлекать сложные признаки из изображений на различных уровнях абстракции. Примеры включают определение объектов на фотографиях, таких как люди, животные или транспортные средства.

2) Распознавание лиц СНС используются для идентификации и верификации лиц в таких системах, как биометрические системы доступа или приложения для маркировки людей на фотографиях в социальных сетях. Они способны обрабатывать изменения в освещении, позе и выражении лица.

3) Анализ и сегментация медицинских изображений. В медицинской диагностике СНС используются для анализа изображений, полученных с помощью МРТ, КТ и других медицинских визуализирующих технологий. СНС помогают выявлять опухоли, аномалии и измерять критические показатели здоровья на изображениях.

4) Анализ видео СНС применяются для распознавания и классификации действий в видео, определения сцен и сегментации движущихся объектов. Это важно для видеонаблюдения, спортивного анализа и автоматизированного видеоконтента.

5) Автоматизированное вождение. В системах автономного вождения СНС используются для обработки визуальной информации, получаемой от камер на автомобиле, что позволяет распознавать дорожные знаки, сигналы светофоров, пешеходов, и другие транспортные средства.

6) Улучшение изображений СНС также используются для улучшения качества изображений, включая удаление шума, увеличение

разрешения и восстановление поврежденных изображений. Это применяется в реставрации старых фотографий и видео, а также в приложениях для улучшения изображений смартфонов.

Эти приложения демонстрируют гибкость и мощь СНС в различных сценариях обработки изображений и видео, делая их неотъемлемой частью современных технологий в области компьютерного зрения. [21]

1.4 Архитектура сверточных нейронных сетей

1.4.1 Слои сверточных нейронных сетей

Сверточные нейронные сети (СНС) имеют характерную архитектуру, которая включает несколько типов слоев, каждый из которых выполняет уникальные функции в процессе обработки и анализа изображений. Ниже описаны основные типы слоев, обычно используемые в СНС: [1]

1) Сверточный слой (Convolutional Layer). Сверточный слой является основой любой СНС и играет ключевую роль в извлечении признаков из изображений. В этом слое происходит применение фильтров (или ядер), которые перемещаются по всему изображению с целью создания карт признаков. Каждый фильтр активно реагирует на определенные визуальные характеристики изображения, такие как края, текстуры или углы. Результатом является несколько карт признаков, каждая из которых соответствует определенному фильтру. [14]

2) Слой подвыборки (Pooling Layer). Слой подвыборки, также известный как слой субдискретизации, следует за одним или несколькими сверточными слоями и служит для уменьшения размерности каждой карты признаков. Это уменьшение размера не только сокращает количество параметров и вычислений в сети, но и делает представление признаков более абстрактным, что увеличивает инвариантность сети к небольшим сдвигам и деформациям во входных данных. Существует несколько видов слоя подвыборки, наиболее распространенные из которых: [21]

2.1) Max Pooling: выбирает максимальное значение из каждого участка карты признаков.

2.2) Average Pooling: вычисляет среднее значение каждого участка карты признаков.

3) Полносвязный слой (Fully Connected Layer). После нескольких чередующихся сверточных и слоев подвыборки, архитектура СНС обычно включает один или несколько полносвязных слоев. Эти слои аналогичны традиционным многослойным перцептронам (MLP) и используются для классификации или регрессии на основе извлеченных и сжатых признаков. В полносвязных слоях каждый нейрон связан со всеми активациями предыдущего слоя, и их основная задача — вывести окончательные выводы о входных данных, такие как классификационные метки. [20]

Эта последовательная комбинация сверточных слоев, слоев подвыборки и полносвязных слоев позволяет СНС эффективно обрабатывать и анализировать визуальные данные. Архитектура сети может быть дополнена другими слоями, такими как нормализация по батчам или слои Dropout для борьбы с переобучением, что делает сети более эффективными и устойчивыми. [22]

1.4.2 Примеры популярных архитектур

Сверточные нейронные сети (СНС) претерпели значительное развитие с момента своего создания, и множество архитектур СНС было разработано для различных задач компьютерного зрения. Некоторые из самых известных и влиятельных архитектур включают LeNet, AlexNet, VGG и ResNet. Вот краткий обзор каждой из них:

1) LeNet

Разработчик: Янн ЛеКун в 1990-х годах.

Особенности: LeNet была одной из первых сверточных нейронных сетей и использовалась в основном для распознавания рукописных цифр и символов в документах. Архитектура LeNet включала чередование сверточных слоев и слоев подвыборки (pooling), заканчиваясь несколькими полносвязными слоями. Эта сеть стала основой для многих последующих разработок в области СНС.

2) AlexNet

Разработчик: Алекс Крижевски, Илья Суцкевер и Джеффри Хинтон, представлена в 2012 году.

Особенности: AlexNet значительно повысила интерес к сверточным нейронным сетям после победы в соревновании ImageNet. Сеть имела глубокую архитектуру из 8 слоев, включая 5 сверточных и 3 полносвязных слоя, и использовала ReLU в качестве функции активации для ускорения обучения. AlexNet также внедрила использование техник, таких как Dropout и Data Augmentation, для улучшения обобщающей способности сети.

3) VGG (VGG16 и VGG19)

Разработчик: Группа исследователей из Оксфордского университета в 2014 году.

Особенности: VGG — это улучшение AlexNet с увеличенной глубиной сети. Архитектура VGG использует маленькие сверточные фильтры (3x3) и включает до 19 слоев (VGG19). Основным преимуществом VGG является то, что она показала, как увеличение глубины сети может существенно улучшить ее производительность.

4) ResNet (Residual Network)

Разработчик: Каймин Хе и коллеги из Microsoft Research, представлена в 2015 году.

Особенности: ResNet решила проблему исчезающих градиентов при обучении очень глубоких нейронных сетей благодаря внедрению "остаточных блоков", которые позволяют передавать входные данные через "пропускные соединения" или "сквозные соединения" на более поздние слои. Это позволило обучать сети, которые значительно глубже предыдущих архитектур, достигая глубины в 152 слоя и более.

Эти архитектуры имели огромное влияние на развитие компьютерного зрения и искусственного интеллекта, позволяя создавать более эффективные и точные системы для анализа визуальной информации. Визуальное

представление некоторых из этих и других архитектур приведено на рисунке 1.6.

Существует многообразие архитектур

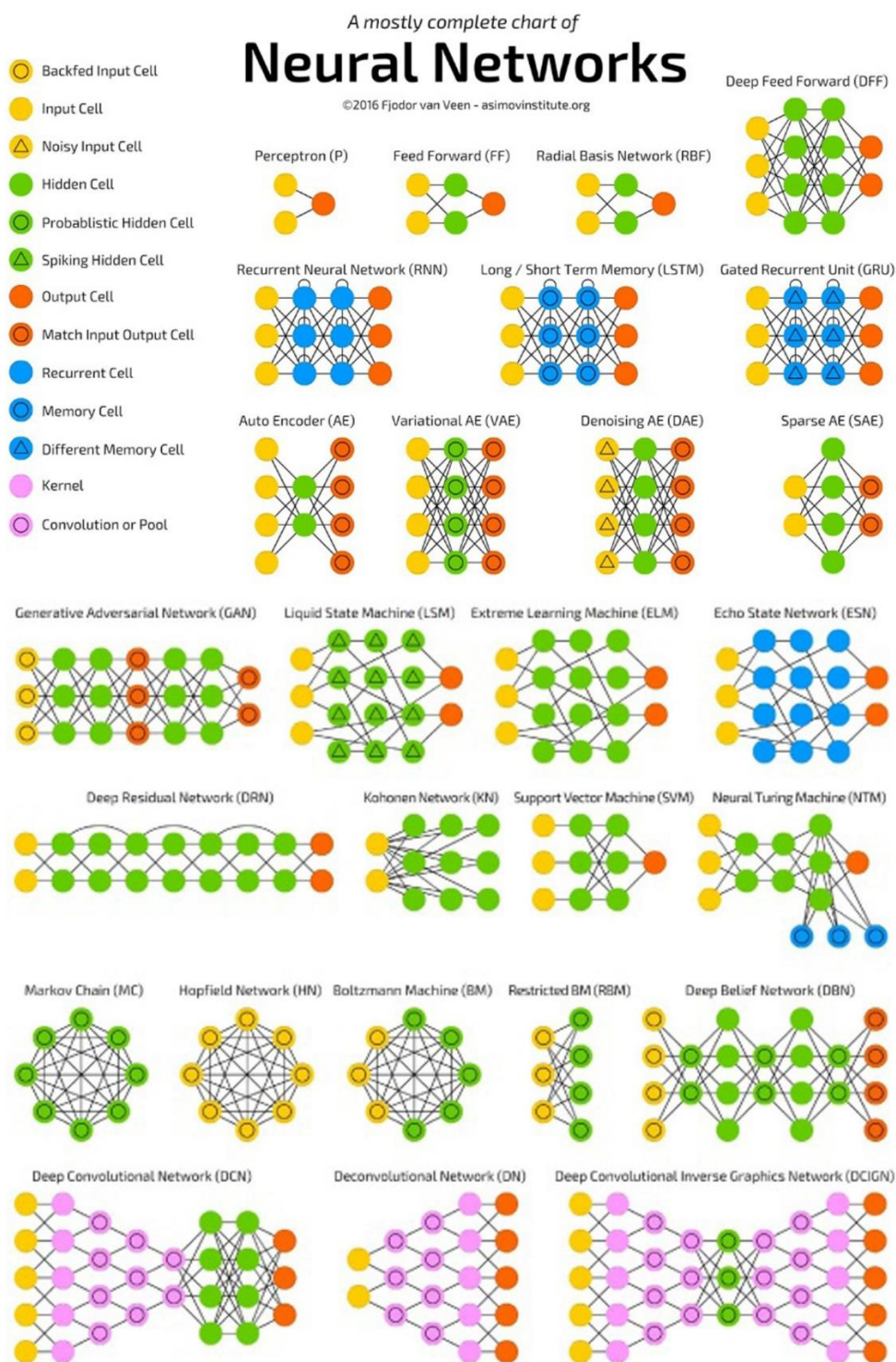


Рисунок 1.6 – Обзор различных архитектур нейронных сетей: перцептроны, сверточные сети (CNN), автоэнкодеры, рекуррентные сети (RNN), остаточные сети (ResNet), генеративные состязательные сети (GAN) и другие.

1.5 Тренировка сверточных нейронных сетей

1.5.1 Подготовка данных и аугментация

Для успешного обучения сверточных нейронных сетей (СНС) крайне важны качественно подготовленные данные и их аугментация. Эти этапы помогают улучшить способность сети к обобщению, что особенно важно при работе с визуальными данными. [1]

Подготовка данных. Перед началом обучения СНС важно правильно подготовить данные. Это включает в себя следующие шаги:

1) Сбор данных: Обеспечение достаточного количества данных для обучения модели. Для компьютерного зрения это может включать сбор большого количества изображений, относящихся к разным категориям. [9]

2) Масштабирование и нормализация: Изображения должны быть приведены к единому размеру, который ожидает входной слой сети. Кроме того, пиксели часто нормализуются так, чтобы их значения находились в определенном диапазоне (например, от 0 до 1 или от -1 до 1), что помогает улучшить стабильность и скорость обучения сети. [3]

3) Разделение данных: Данные обычно разделяются на обучающую, валидационную и тестовую выборки. Обучающая выборка используется для тренировки сети, валидационная — для настройки параметров, а тестовая — для оценки окончательной производительности модели. [23]

Аугментация данных — это техника, которая позволяет значительно увеличить разнообразие обучающих данных без необходимости собирать новые изображения. Это достигается путем применения случайных, но реалистичных изменений к обучающим изображениям (пример приведен на рисунке 1.7). Вот некоторые популярные методы аугментации для изображений: [24]

1) Вращение: Изображение поворачивается на случайный угол. Это помогает модели учиться распознавать объекты независимо от их ориентации.

2) Масштабирование: Изменение размера изображения помогает сети учиться определять объекты разных размеров.

3) Горизонтальное и вертикальное отражение: Зеркальное отображение изображений помогает модели не зависеть от того, в какой части изображения находится объект.

4) Изменение яркости и контраста: Модификация этих параметров может помочь сети быть менее чувствительной к условиям освещения.

5) Сдвиг: Сдвиг изображения вверх, вниз, влево или вправо помогает учиться распознавать объекты, даже если они частично выходят за границы поля зрения.

6) Шум: Добавление случайного шума к изображениям может помочь модели устойчиво работать в условиях низкого качества изображений. [25]

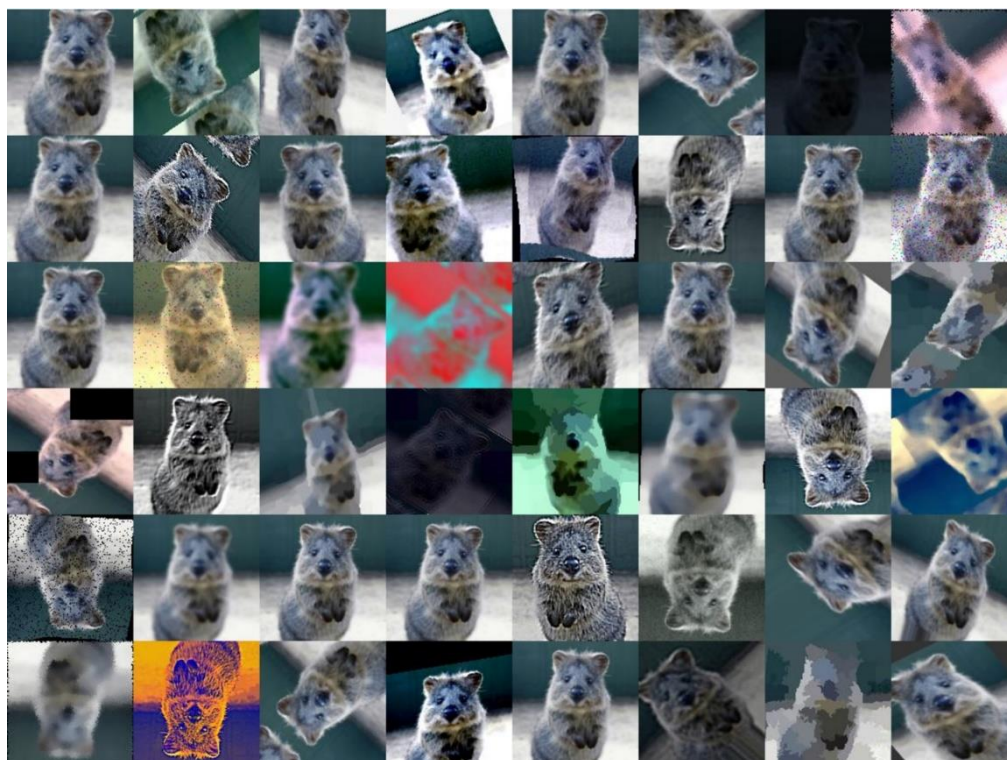


Рисунок 1.7 – Пример аугментации данных: одно исходное изображение модифицировано различными методами, включая повороты, шум, отражения, фильтры, искажения и изменения контраста.

Аугментация данных является мощным инструментом для повышения устойчивости и обобщающей способности сверточных нейронных сетей,

позволяя им более эффективно работать с реальными, часто неполными или зашумленными данными

1.5.2 Проблемы переобучения и методы их решения, такие как Dropout, регуляризация

Переобучение (overfitting) — это явление, при котором нейронная сеть слишком точно подстраивается под данные обучения, в результате чего её способность к обобщению на новых данных ухудшается. Объяснение переобучения на пальцах: «Ты не выучил тему — ты просто выучил, где какие ответы в учебнике». Это одна из основных проблем при тренировке сверточных нейронных сетей, особенно когда доступное количество обучающих данных ограничено или когда архитектура сети слишком сложная по сравнению с количеством и разнообразием обучающих данных.

Методы решения проблемы переобучения:

1) Dropout — это техника регуляризации, при которой случайным образом исключаются (обнуляются) некоторые элементы нейронной сети во время тренировки. Это помогает предотвратить чрезмерную зависимость сети от любых отдельных элементов или пути данных и способствует лучшей обобщаемости. Dropout обычно применяется в полносвязных слоях, так как они более склонны к переобучению, но может использоваться и в сверточных слоях.

2) Регуляризация добавляет штраф к функции потерь сети за слишком большие веса, что ограничивает их рост и способствует более гладкой и устойчивой модели. Существуют различные виды регуляризации:

2.1) L1 регуляризация (регуляризация Лассо): добавляет штраф, равный абсолютной величине величины параметров.

2.2) L2 регуляризация (регуляризация Ридж): добавляет штраф, равный квадрату величины параметров. Это самая часто используемая форма регуляризации в нейронных сетях и помогает контролировать переобучение, делая веса более маленькими.

3) Ранняя остановка (Early Stopping). Этот метод заключается в мониторинге производительности модели на валидационном наборе данных в процессе обучения и прекращении обучения, как только производительность перестает улучшаться. Это предотвращает дальнейшее обучение модели, которое могло бы привести к переобучению.

4) Уменьшение сложности модели. Простые модели меньше подвержены переобучению. Если сеть переобучается, можно попробовать уменьшить количество слоев или количество нейронов в слоях, чтобы упростить модель.

5) Увеличение количества данных. Чем больше данных доступно для обучения, тем лучше модель сможет обобщать на новые данные. Кроме увеличения исходного набора данных, можно использовать техники аугментации данных для искусственного увеличения количества и разнообразия обучающих примеров.

6) Использование Batch Normalization. Batch Normalization — это техника, которая нормализует входы каждого слоя по мини-батчам, что помогает ускорить обучение сети и уменьшает вероятность переобучения.

Применение этих методов может значительно повысить обобщающую способность сверточных нейронных сетей и улучшить их производительность на новых, ранее не виденных данных.

1.6 Примеры использования СНС в реальных приложениях

1.6.1 Распознавание изображений. Анализ видео

Анализ видео — ещё одна область, где СНС демонстрируют впечатляющие результаты, обеспечивая глубокое понимание содержания видео:

1) Безопасность и наблюдение: СНС используются для мониторинга видео в реальном времени, чтобы определять аномальные поведения или события, что критично для систем безопасности на объектах или в общественных местах. [26]

2) Спортивный анализ: В спорте СНС применяют для анализа видео игр, чтобы выявлять стратегии, отслеживать движения игроков и автоматически генерировать основные моменты игры для последующего просмотра. [27]

3) Автоматизация транспортных средств: В области автономных транспортных средств СНС анализируют видеопоток для навигации и управления транспортным средством, распознавая дорожные знаки, сигналы светофора и препятствия на дороге. [28]

Эти примеры показывают, как СНС могут преобразовывать массивные и сложные визуальные данные в полезную информацию, что делает их неотъемлемой частью многих современных технологических систем. Наличие больших объемов данных и мощных вычислительных ресурсов продолжает способствовать развитию и интеграции СНС в ещё большее количество приложений.

1.6.2 Медицинская диагностика

Применение сверточных нейронных сетей (СНС) в медицинской диагностике.

Сверточные нейронные сети (СНС) стали крайне важным инструментом в медицинской диагностике, предоставляя новые возможности для анализа медицинских изображений с высокой точностью. Использование СНС в этой области помогает улучшить качество и скорость диагностики, снизить нагрузку на медицинских специалистов и повысить общую доступность медицинских услуг. Вот несколько ключевых примеров применения СНС в медицине:

1) Диагностика рака. Распознавание опухолей на маммограммах: СНС могут анализировать маммограммы на предмет наличия аномальных образований, таких как опухоли, что помогает в раннем обнаружении рака молочной железы. [29, 30]

Анализ гистопатологических изображений: СНС используются для анализа образцов тканей, полученных при биопсии, для точной диагностики рака и других заболеваний.

2) Нейроимиджинг. Диагностика исследований мозга: СНС применяются для анализа изображений МРТ и КТ головного мозга, помогая выявлять признаки неврологических заболеваний, таких как Альцгеймера, инсульты и эпилепсия. [31]

3) Анализ снимков грудной клетки. Обнаружение пневмонии и туберкулеза: СНС могут обнаруживать признаки пневмонии и туберкулеза на рентгеновских снимках грудной клетки, что критично для своевременного лечения этих потенциально опасных заболеваний.

4) Офтальмология. Диагностика заболеваний глаз: СНС эффективно используются для анализа изображений сетчатки глаза, диагностики диабетической ретинопатии, возрастной макулярной дегенерации и других заболеваний, которые могут привести к ухудшению зрения или слепоте. [32]

5) Кардиология. Анализ сердечно-сосудистых изображений: СНС применяются для анализа ультразвуковых изображений сердца (эхокардиограмм) и других сердечно-сосудистых исследований для диагностики заболеваний, таких как ишемическая болезнь сердца и сердечная недостаточность. [33]

Применение СНС в медицинской диагностике не только улучшает точность и эффективность диагностических процессов, но и открывает новые возможности для раннего обнаружения заболеваний и индивидуализированного подхода к лечению пациентов. Эти технологии способствуют переходу от реактивной медицины к проактивной, что может существенно улучшить качество жизни и прогнозы для множества пациентов.

1.7 Тенденции и будущее сверточных нейронных сетей

1.7.1 Новые направления в архитектуре СНС

Сверточные нейронные сети (СНС) продолжают развиваться, благодаря постоянным исследованиям и инновациям в области глубокого обучения и

искусственного интеллекта. Новые направления в архитектуре СНС включают улучшения в производительности, эффективности и способности обобщения. Вот несколько ключевых тенденций и будущих направлений в развитии СНС:

1) Эффективность и оптимизация. Современные исследования СНС направлены на создание более эффективных моделей, которые требуют меньше вычислительных ресурсов и могут работать на устройствах с ограниченными возможностями, таких как мобильные телефоны и встраиваемые системы. Например, разработки такие как MobileNet, EfficientNet и SqueezeNet демонстрируют, как можно сократить количество параметров сети без значительной потери в точности. [34]

2) Автоматическое проектирование нейронных сетей (Neural Architecture Search, NAS). NAS — это технология, позволяющая автоматически находить оптимальные архитектуры сетей для конкретных задач. Этот подход использует алгоритмы машинного обучения для итеративного тестирования и сравнения различных архитектур сетей, что позволяет разработчикам сосредоточиться на задачах более высокого уровня. [35]

3) Интеграция с механизмами внимания. СНС все чаще интегрируются с механизмами внимания, аналогичными тем, что используются в трансформерах. Механизмы внимания позволяют моделям более эффективно выделять информативные части данных, что приводит к улучшению производительности в таких задачах, как обработка естественного языка и сложные задачи компьютерного зрения. [12]

4) Глубокое обучение с подкреплением. Использование СНС в сочетании с алгоритмами глубокого обучения с подкреплением открывает новые возможности в таких областях, как автоматическое вождение и робототехника. Это позволяет моделям не только воспринимать своё окружение, но и принимать решения в реальном времени на основе непрерывного обучения. [36]

5) Объединение и слияние моделей. Разработка гибридных моделей, которые объединяют СНС с другими видами нейронных сетей, такими как рекуррентные нейронные сети (RNN) и генеративно-сопоставительные сети (GAN), предоставляет новые возможности для создания более мощных и универсальных систем ИИ. Это особенно актуально в мультизадачных системах, где одна модель должна справляться с различными типами данных и задачами. [37]

6) Усиление приватности и безопасности. По мере того, как СНС все чаще используются в чувствительных областях, таких как медицина и финансы, вопросы безопасности и приватности данных становятся всё более актуальными. Исследования направлены на разработку методов, которые могут обеспечивать защиту данных при сохранении производительности моделей. [38]

Эти тенденции подчеркивают динамичное развитие сверточных нейронных сетей и их важность в будущем технологий искусственного интеллекта.

1.7.2. Проблемы и перспективы в развитии сверточных нейронных сетей

Сверточные нейронные сети (СНС) продолжают преобразовывать множество отраслей, но их развитие сопряжено как с проблемами, так и с возможностями для дальнейших инноваций. Ниже представлены некоторые из ключевых проблем и перспектив, связанных с развитием и применением СНС.

Проблемы:

1) Вычислительные затраты: Обучение СНС требует значительных вычислительных ресурсов, особенно для больших моделей и больших наборов данных. Это может ограничивать доступность и масштабирование этих технологий, особенно в условиях с ограниченными ресурсами.

2) Нужды в данных: СНС требуют больших объемов маркированных данных для эффективного обучения. Сбор и аннотация

данных могут быть дорогостоящими и времязатратными, а в некоторых случаях — невозможными из-за проблем конфиденциальности или доступности.

3) Переобучение: хотя существуют методы борьбы с переобучением, такие как регуляризация и Dropout, это всё ещё остается значительной проблемой, особенно когда данные ограничены или очень разнообразны.

4) Проблемы с приватностью и этикой: Использование СНС в таких областях, как распознавание лиц и анализ личных данных, порождает вопросы о конфиденциальности, согласии и этическом использовании искусственного интеллекта.

5) Объяснимость и прозрачность: СНС часто критикуют за их "черный ящик" характер, который затрудняет понимание того, как именно принимаются решения. Это особенно важно в критичных приложениях, таких как медицина и автоматизированное вождение.

Перспективы:

Улучшение алгоритмов: Исследования по оптимизации алгоритмов и разработке более эффективных моделей продолжаются. Прогресс в этой области может помочь снизить вычислительные затраты и улучшить доступность СНС.

1) Синтетические и аугментированные данные: Развитие технологий генерации синтетических данных и методов аугментации может помочь преодолеть нехватку маркированных данных и улучшить обучение моделей.

2) Нейронные сети нового поколения: Разработка новых архитектур, таких как капсульные сети и спайковые нейронные сети, обещает улучшения в плане обработки данных и принятия решений.

3) Повышение прозрачности и интерпретируемости: Разработка инструментов для визуализации и интерпретации внутренней работы СНС может помочь сделать эти модели более понятными и надежными.

4) Этические и законодательные инициативы: Продолжение разработки этических руководств и законодательных рамок для

использования искусственного интеллекта может помочь обеспечить его безопасное и ответственное применение.

Исследования и разработки в области СНС продолжают оставаться активными и динамичными, с обещанием значительных улучшений в технологии и ее применении. Эти технологии обладают потенциалом для радикального изменения множества аспектов нашей жизни и работы, предлагая как значительные возможности, так и вызовы.

2 СПЕЦИАЛЬНАЯ ЧАСТЬ

2.1 Постановка задачи классификации

Классификация изображений — это одна из ключевых задач в области машинного обучения и компьютерного зрения, которая направлена на распределение входных данных по заданным категориям. В последние годы задачи классификации нашли широкое применение в промышленности, медицине, науке и других областях, где необходима автоматизация процесса распознавания объектов, явлений или состояний. аппаратам (БПЛА) задачи классификации приобрели особое значение, так как позволяют решать вопросы диагностики состояния техники и обнаружения потенциально опасных дефектов, что в конечном итоге обеспечивает безопасную эксплуатацию и повышает надежность авиационных систем. На рисунке 2.1 показана схема применения БПЛА, оснащенного камерой, для визуального осмотра поверхности воздушного судна. [1, 3]

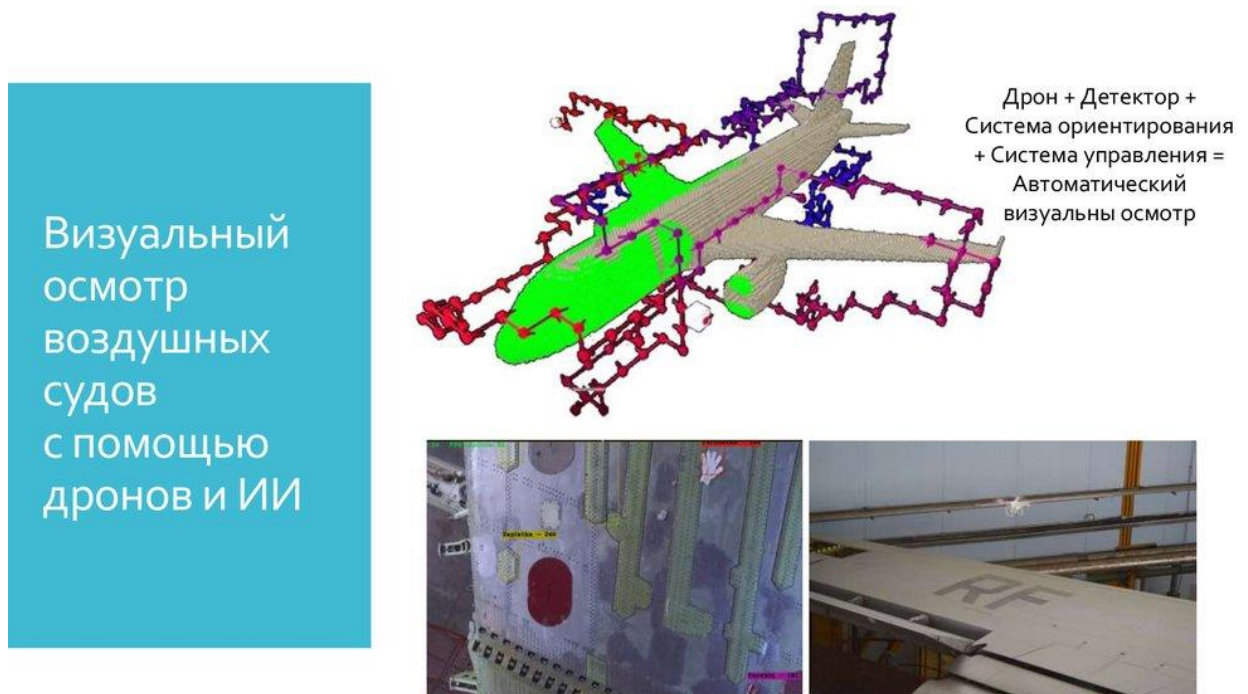


Рисунок 2.1 – Схема применения БПЛА с камерой и системой управления для автоматизированного визуального осмотра воздушного судна используя методы аугментации, которые позволяют модели.

Традиционные методы диагностики дефектов на воздушных судах включают визуальный осмотр, ультразвуковую и радиографическую диагностику, а также другие неразрушающие методы контроля. Однако эти подходы требуют значительных человеческих ресурсов и временных затрат, так как для выполнения каждого осмотра требуется квалифицированный персонал и специализированное оборудование. Более того, такие методы обладают ограниченной способностью обнаруживать мелкие или скрытые дефекты, особенно в условиях труднодоступных для осмотра областей. В результате возникает необходимость создания систем автоматизированной диагностики, которые могли бы анализировать изображения и обнаруживать дефекты с высокой точностью. [4, 29]

Важной частью разработки системы автоматизированного контроля на основе БПЛА является задача классификации полученных изображений дефектов. Автоматическое определение и классификация повреждений помогают минимизировать влияние человеческого фактора и ускоряют процесс принятия решений. Для реализации такого подхода используются методы машинного обучения, в частности, нейронные сети, которые способны обрабатывать сложные визуальные данные и выделять ключевые признаки, указывающие на тип дефекта. [1]

Целью данного исследования является разработка модели нейронной сети, способной классифицировать различные типы повреждений на основе изображений, полученных с камер, установленных на БПЛА. Данная система автоматической классификации должна удовлетворять ряду требований, включая высокую точность, способность работать с реальными изображениями, полученными в полевых условиях, а также устойчивость к различным условиям съемки (разные углы, освещение и т.д.). Подобная система может стать важным компонентом для обеспечения своевременного обнаружения дефектов, что позволит снизить эксплуатационные риски и обеспечить высокую надежность летательных аппаратов. [3, 30]

Создание такой системы классификации требует решения ряда задач:

1) Обработка и подготовка данных. В первую очередь необходимо собрать и подготовить изображения, представляющие различные типы дефектов. Эти данные будут использоваться как для обучения модели, так и для её тестирования. [18]

2) Создание архитектуры нейронной сети. Нужно построить архитектуру, способную выделять признаки и осуществлять классификацию с учетом сложности и разнообразия входных данных. [21]

3) Аугментация данных. Поскольку в реальных условиях БПЛА могут работать при разных освещениях, углах съемки и фонах, требуется создать разнообразие тренировочных данных, используя методы аугментации, которые позволяют модели адаптироваться к различным условиям (см. рисунок 2.2). [23]

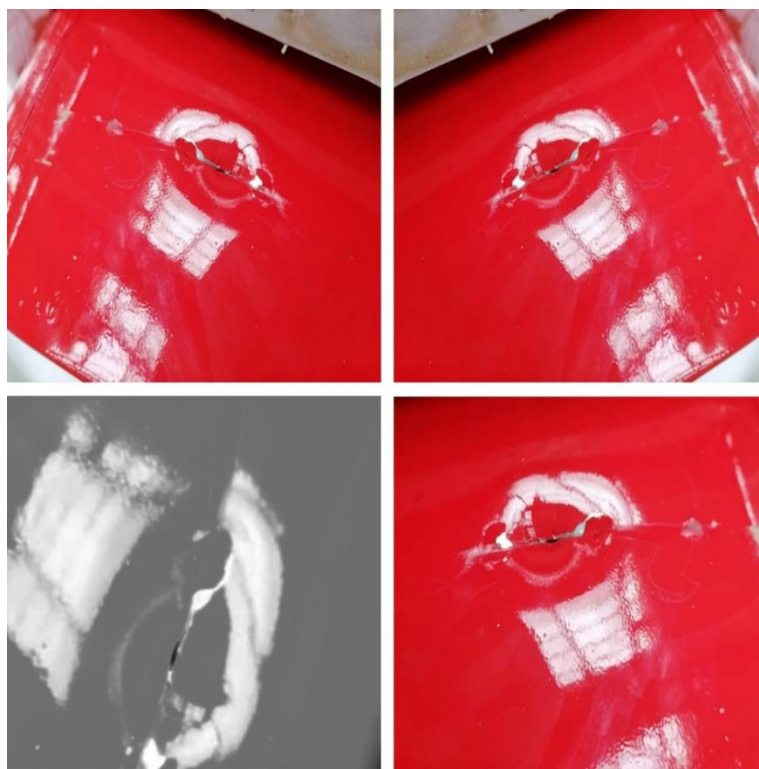


Рисунок 2.2. – Примеры аугментации изображений: повороты, отражения, масштабирование, искажения и фильтрация, применяемые для повышения разнообразия обучающего набора.

4) Процесс обучения и валидации. Модель должна пройти процесс обучения и валидации, чтобы достичь приемлемого уровня точности, который позволит использовать её на практике. [15]

5) Оценка качества модели. Для подтверждения эффективности модели требуется провести оценку на тестовых данных, а также сравнить её с другими архитектурами нейронных сетей, чтобы выбрать наиболее подходящий подход. [20]

Предложенная система классификации должна решать задачу многоклассовой классификации, так как изображения дефектов будут разделяться на три основные категории: вмятины (Dent), повреждение крепежных элементов (Fastener Damage) и разрывы (Rupture). Модель должна научиться выделять специфические признаки для каждой категории дефектов, чтобы классифицировать изображения с высокой точностью. [26]

Таким образом, основная задача классификации дефектов на БПЛА в рамках данного исследования заключается в создании модели, способной обрабатывать изображения повреждений и относить их к одному из заданных классов дефектов. В отличие от других подходов, система на основе нейронных сетей обладает высокой гибкостью, так как может работать в условиях ограниченного освещения, различных углов обзора и других факторов, которые часто влияют на качество диагностики. [1]

Применение нейронных сетей для классификации дефектов открывает широкие перспективы в области диагностики БПЛА. Система классификации на базе нейронной сети способна обрабатывать большое количество изображений за короткое время, что делает её незаменимым инструментом для анализа состояния техники и принятия своевременных мер по устранению дефектов. [19]

2.2 Описание данных

Для построения и обучения модели классификации в данном исследовании был использован набор изображений повреждений различных частей летательных аппаратов, полученных с камер, установленных на беспилотных летательных аппаратах (БПЛА). Эти данные были специально собраны для автоматического анализа и классификации дефектов, что делает их особенно полезными для разработки системы диагностики. [30]

2.2.1 Входные данные

Формат и структура изображений. Все изображения, используемые для обучения и оценки модели, имеют фиксированное разрешение 640x640 пикселей и представлены в цветовой схеме RGB (Red-Green-Blue). Высокое разрешение и наличие цветовой информации позволяют модели учитывать все визуальные признаки, что повышает её способность к точной классификации. Изображения были предварительно обработаны для унификации размеров, что упрощает дальнейшую работу с ними и помогает стандартизировать процесс подачи данных в модель. [4]

2.2.2 Типы дефектов и классы

В рамках данного исследования классифицируются три основных типа дефектов, которые могут возникнуть на различных частях летательного аппарата:

- 1) Вмятины (Dent) — это небольшие деформации, которые могут появляться на поверхности летательного аппарата в результате механических воздействий. Вмятины могут казаться малозначительными, но в некоторых случаях они могут быть маркером более глубоких структурных повреждений. Пример такой деформации представлен на рисунке 2.3.



Рисунок 2.3 – Вмятина (Dent) на конструкции летательного аппарата, возникшая в результате механического воздействия.

2) Повреждения крепежных элементов (Fastener Damage) — дефекты, связанные с повреждением или нарушением целостности крепежных элементов, таких как болты, винты и другие соединительные элементы. Эти повреждения могут оказывать значительное влияние на прочность конструкции и должны быть своевременно обнаружены. На рисунке 2.4 показан пример повреждения крепежных элементов. [31]



Рисунок 2.4 – Повреждение крепежного элемента (Fastener Damage), сопровождающееся деформацией области вокруг крепления.

3) Разрывы (Rupture) — крупные повреждения, представляющие собой трещины или разрывы в обшивке или других структурных элементах летательного аппарата. Разрывы являются самыми опасными повреждениями и требуют немедленного вмешательства для предотвращения аварийных ситуаций. Визуальный пример разрыва конструкции приведен на рисунке 2.5. [30]

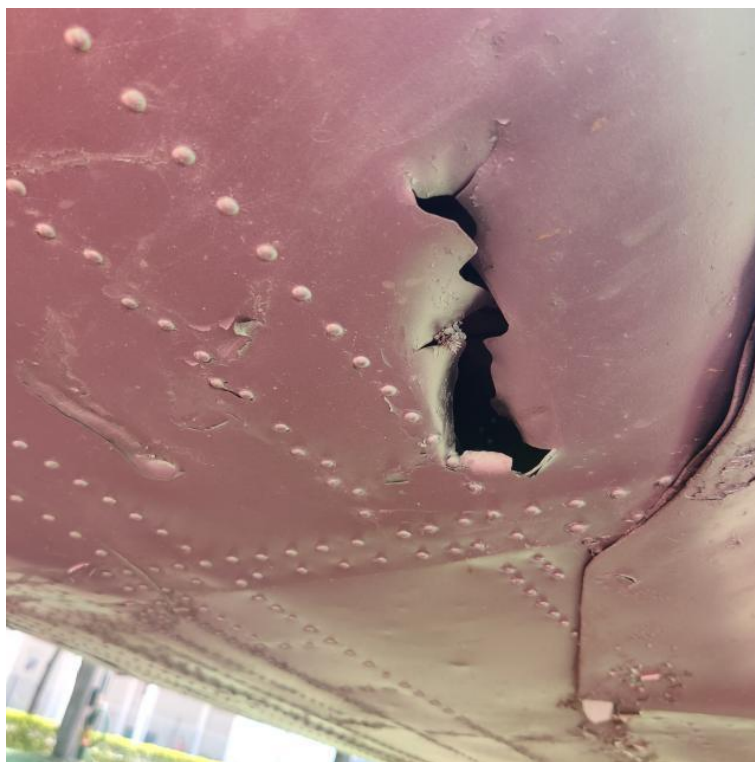


Рисунок 2.5 – Разрыв (Rupture) в обшивке, характеризующийся значительным нарушением целостности материала.

Каждая из перечисленных категорий дефектов выделяется специфическими визуальными признаками, которые используются моделью для выполнения классификации. Таким образом, набор данных включает три класса, каждый из которых отражает определенный тип повреждения, что позволяет решать задачу многоклассовой классификации. [3]

2.2.3 Объем данных

Общий объем набора данных составил 9300 изображения, что обеспечивает достаточное количество примеров для обучения, валидации и тестирования модели. Изображения были разделены на подмножества:

- 1) Обучающая выборка — основной набор изображений, используемый для настройки весов и обучения модели.
- 2) Валидационная выборка — используется для оценки производительности модели на изображениях, не включенных в обучение, с целью предотвращения переобучения.
- 3) Тестовая выборка — применяется для окончательной оценки качества модели после завершения этапа обучения. Этот набор позволяет

проверить способность модели классифицировать ранее не виденные изображения и определить, насколько эффективно она будет работать в реальных условиях. Визуальное распределение изображений по обучающей, валидационной и тестовой выборкам представлено на рисунке 2.6. [18]



Рисунок 2.6 – Диаграмма, иллюстрирующая распределение 9300 изображений по трем подвыборкам: обучающей (7500), валидационной (900) и тестовой (900), по 2500, 300 и 300 изображений соответственно на каждый из трех классов дефектов.

2.2.4 Алфавит классов

Для упрощения процесса классификации и интерпретации результатов каждому классу было присвоено уникальное название:

- 1) Dent — вмятины;
- 2) Fastener Damage — повреждения крепежных элементов;
- 3) Rupture — разрывы.

Такой алфавит классов используется моделью в процессе классификации для присвоения предсказаниям каждого изображения одного из этих трех типов дефектов. В дальнейшем это позволяет легко

анализировать результаты классификации и выявлять долю правильно классифицированных изображений для каждого типа повреждений. [26]

2.2.5 Реалистичность условий съемки

Для достижения максимально возможной точности в реальных условиях эксплуатации данных для диагностики, изображения, собранные для этого исследования, охватывают широкий спектр условий, отражающих возможные сценарии полевой эксплуатации. Набор данных включает изображения, снятые при различных:

1) Углах обзора. Изображения дефектов были получены с разных углов, что делает задачу классификации сложной, но одновременно приближает её к реальным условиям, где камеры БПЛА могут снимать объект под разными углами.

2) Условиях освещения. Съемка выполнялась при различной яркости и освещенности, что также затрудняет классификацию, так как дефекты могут выглядеть по-разному в зависимости от освещения.

3) Ракурсах и фонах. Некоторые изображения имеют различный фон или объекты на заднем плане, что делает задачу классификации более сложной, так как модель должна научиться выделять ключевые признаки дефектов, игнорируя нерелевантные детали фона. [23, 28]

Наличие подобных вариаций делает задачу классификации не только более сложной, но и более ценной с точки зрения приближенности к реальной эксплуатации. Модель, обученная на таком наборе данных, становится менее чувствительной к изменению условий съемки и способна выполнять классификацию в широком диапазоне сценариев. [13]

2.2.6 Предобработка данных

Для подготовки данных к обучению использовались методы нормализации и аугментации, позволяющие улучшить качество модели и её способность к обобщению. Все изображения были нормализованы, то есть значения пикселей были преобразованы в диапазон от 0 до 1, что помогает улучшить работу модели и избежать переобучения. [23]

Также для обучающей выборки была использована аугментация данных, которая позволила виртуально увеличить объем данных, создавая измененные версии изображений (подробнее описана в пункте 2.3). [24]

Таким образом, описанный набор данных обеспечивает достаточную базу для эффективного обучения нейронной сети, что позволит модели классифицировать изображения дефектов с высокой точностью в различных эксплуатационных условиях.

2.3 Методы аугментации данных

Аугментация данных — это процесс создания новых примеров на основе существующих изображений путем их преобразований и изменений, которые сохраняют основные характеристики, но добавляют вариативность. В задачах классификации изображений аугментация является одним из основных методов повышения качества и устойчивости моделей машинного обучения. Она особенно важна в тех случаях, когда объем доступных данных ограничен или требуется, чтобы модель могла справляться с разнообразными условиями, как это происходит в реальной эксплуатации. [23, 39]

Для задачи классификации дефектов на БПЛА аугментация данных позволила увеличить разнообразие тренировочного набора, что помогло модели стать более гибкой и устойчивой к изменениям в условиях съемки, таким как освещение, ракурс и фон. В данном исследовании использовались следующие методы аугментации данных:

2.3.1 Повороты изображений (Rotation)

В процессе съемки дефектов с беспилотных летательных аппаратов изображения могут быть получены под различными углами. Это особенно актуально для БПЛА, так как в зависимости от их положения в воздухе и угла камеры, снимки могут быть сделаны с различным наклоном. [23, 24]

Чтобы имитировать эту вариативность, изображения подвергались случайным поворотам на угол до 180 градусов. Такой подход позволяет улучшить устойчивость модели к наклонам и положениям объекта на изображении, что помогает избежать ошибки в классификации дефектов,

которые имеют похожие признаки, но сняты под разными углами. На рисунке 2.7 приведён пример изображения с дефектом, подвергнутого повороту под различными углами. Повороты помогают модели не привязываться к фиксированной ориентации объектов, повышая её способность к обобщению.



Рисунок 2.7 – Пример аугментации с использованием поворота: одно и то же изображение объекта с дефектом, представлено в разных ориентациях для имитации возможных углов съёмки с БПЛА..

2.3.2 Масштабирование (Zooming)

Размер дефектов, таких как вмятины, разрывы и повреждения крепежных элементов, может значительно варьироваться. Например, некоторые повреждения могут занимать большую часть изображения, тогда как другие дефекты могут быть мелкими и едва заметными.

Для учета этой вариативности использовалось масштабирование изображений. Оно включало в себя как увеличение, так и уменьшение изображений на случайную величину, что позволило модели распознавать дефекты разных размеров. На рисунке 2.8 показано изображение, подвергнутое увеличению и уменьшению, что имитирует разные дистанции съёмки с БПЛА. Масштабирование также помогло улучшить устойчивость модели к различной удаленности объекта на изображении, так как БПЛА может снимать повреждения с разных расстояний. [6]



Рисунок 2.8 – Пример масштабирования изображения: увеличение (справа) и уменьшение (слева) изображения дефекта, имитирующее съёмку с разных расстояний.

2.3.3 Горизонтальное отражение (Horizontal Flip)

Благодаря горизонтальному отражению изображения модель обучалась распознавать дефекты в различных ориентациях. Этот метод особенно полезен в условиях, когда объект съёмки (например, часть обшивки летательного аппарата) может находиться в различных положениях относительно камеры.

Отражение изображений по горизонтали добавляет симметричность в тренировочный набор, что помогает модели избегать переобучения на определенные стороны объектов. При этом модель лучше адаптируется к вариативности ориентаций объектов на изображениях, делая её более универсальной и устойчивой к различным условиям эксплуатации. Пример такого горизонтального отражения приведён на рисунке 2.9. [21]



Рисунок 2.9 – Пример горизонтального отражения: одно и то же изображение дефекта зеркально отображено для увеличения симметрии обучающего набора.

2.3.4 Изменение яркости и контрастности (Brightness and Contrast Adjustment)

Условия освещения могут значительно варьироваться в зависимости от времени суток, погодных условий и интенсивности солнечного света. При диагностике с использованием БПЛА камера может фиксировать объект в условиях недостаточного или чрезмерного освещения, что затрудняет распознавание дефектов.

Изменение яркости и контрастности помогает модели справляться с этими вариациями, поскольку она обучается видеть изображения с разной освещенностью. Это делает модель более устойчивой к изменениям внешних условий и позволяет ей лучше адаптироваться к реальным сценариям, в которых БПЛА может фиксировать повреждения при различных уровнях освещения. Пример таких изображений с варьирующейся яркостью и контрастностью приведён на рисунке 2.10. [5]



Рисунок 2.10 – Пример аугментации изображений путём изменения яркости и контрастности: слева – исходное изображение, справа – с пониженной яркостью и контрастом.

2.3.5 Сдвиг и деформация (Shifting and Shearing)

Хотя данный метод менее значим в контексте классификации дефектов, небольшие сдвиги и деформации изображений помогают модели быть менее чувствительной к положению дефекта на изображении. Это полезно, когда точное расположение повреждения в кадре может не совпадать, особенно если учесть, что камера БПЛА не всегда фиксирует повреждения точно в центре кадра.

Сдвиг и деформация изображений создают дополнительные варианты расположения дефектов, что повышает обобщающую способность модели. Этот метод позволяет нейронной сети сосредоточиться на признаках дефектов, а не на их точном расположении на изображении, что делает её более устойчивой к различным позициям объектов. [7]

2.3.6 Суммарная роль аугментации данных

Аугментация данных сыграла важную роль в улучшении обобщающей способности модели, так как позволила увеличить объем данных и внести вариативность в тренировочный набор. Это привело к улучшению качества модели и её устойчивости к различным внешним факторам. [23, 24]

Методы аугментации были применены только к обучающей выборке, поскольку основной целью является улучшение адаптивности модели, тогда как валидационные и тестовые данные оставались неизменными для обеспечения объективной оценки её производительности. В процессе аугментации использовалась библиотека ImageDataGenerator из TensorFlow для генерации измененных изображений в реальном времени, что также помогло оптимизировать процесс и снизить затраты на хранение данных. [13]

Таким образом, аугментация данных помогла создать более гибкую и точную модель, которая способна работать в условиях реальной эксплуатации и справляться с вариациями в условиях съемки, которые могут возникать при диагностике дефектов на БПЛА.

2.4 Построение нейронной сети

В данной работе для решения задачи классификации дефектов на изображениях, полученных с беспилотных летательных аппаратов (БПЛА), была разработана сверточная нейронная сеть (CNN), реализованная с использованием библиотеки TensorFlow и её интерфейса Keras. [1, 6]

Ниже приведен код, описывающий архитектуру нейронной сети, а также подробное объяснение её основных слоев и гиперпараметров.

2.4.1 Параметры модели и пути к данным

Первым шагом была настройка базовых параметров, таких как размер входного изображения, количество эпох, размер батча и количество классов, к которым будет относиться каждая категория дефектов. Эти параметры заданы в коде, представленном на рисунке 2.11. [39]

```
# Параметры
img_width, img_height = 640, 640
batch_size = 32
epochs = 50 # количество эпох для обучения
num_classes = 3 # количество классов дефектов
```

Рисунок 2.11 – Пример задания базовых гиперпараметров модели: размер входного изображения, размер мини-выборки (batch size), количество эпох и число классов дефектов.

Эти параметры задают разрешение изображений, размер мини-выборки для обучения, количество эпох и число классов, в нашем случае — три категории: Dent (вмятины), Fastener Damage (повреждения крепежных элементов) и Rupture (разрывы).

Далее были определены пути к директориям с данными, что обеспечивает корректную организацию структуры проекта и автоматическую загрузку изображений на соответствующих этапах обучения и валидации (рисунок 2.12).

```
data_dir = 'data'
train_data_dir = os.path.join(data_dir, 'train')
validation_data_dir = os.path.join(data_dir, 'valid')
test_data_dir = os.path.join(data_dir, 'test')
```

Рисунок 2.12 – Структура директорий для обучающего, валидационного и тестового наборов данных.

Определение директорий для обучающих, валидационных и тестовых данных

Эти директории хранят обучающие, валидационные и тестовые изображения, что позволяет разделить данные для разных этапов обучения и оценки модели. [7]

2.4.2 Архитектура нейронной сети

Архитектура модели создаётся с помощью класса `Sequential`, позволяющего последовательно добавлять слои. Основные компоненты сверточной нейронной сети, реализованные в коде (см. рисунок 2.13), включают входной слой, три сверточных блока с объединяющим слоем (`MaxPooling`), полносвязный слой и механизм регуляризации `Dropout`:

```
model = Sequential([
    # Входной слой с заданными размерами изображения
    Input(shape=(img_width, img_height, 3)),

    # Первый сверточный слой с L2-регуляризацией
    Conv2D(32, (3, 3), activation='relu',
kernel_regularizer=l2(0.01)),

    # Пуллинг для уменьшения размера карты признаков
    MaxPooling2D(pool_size=(2, 2)),

    # Второй сверточный слой
    Conv2D(64, (3, 3), activation='relu',
kernel_regularizer=l2(0.01)),
    MaxPooling2D(pool_size=(2, 2)),

    # Третий сверточный слой
    Conv2D(128, (3, 3), activation='relu',
kernel_regularizer=l2(0.01)),
    MaxPooling2D(pool_size=(2, 2)),

    # Преобразование карты признаков в одномерный вектор
    Flatten(),
```

```

# Полносвязный слой с L2-регуляризацией
Dense(256, activation='relu',
kernel_regularizer=l2(0.01)),

# Dropout для предотвращения переобучения
Dropout(0.5),

# Выходной слой с softmax для классификации
Dense(num_classes, activation='softmax')
])

```

Рисунок 2.13 – Реализация архитектуры сверточной нейронной сети на базе

Keras: включает входной слой, три сверточных блока, слой Flatten, полносвязный слой и выход с функцией активации softmax.

Рассмотрим каждый из слоев этой архитектуры:

1) Входной слой `Input(shape=(img_width, img_height, 3))` определяет размер входных изображений `(img_width, img_height, 3)`, где 3 соответствует трем каналам RGB.

2) Первый сверточный слой `Conv2D(32, (3, 3), activation='relu', kernel_regularizer=l2(0.01))` содержит 32 фильтра размером 3x3. Он использует ReLU в качестве функции активации, что позволяет сети учитывать только значимые признаки. Применяется L2-регуляризация (`l2(0.01)`), которая штрафует слишком большие веса, помогая избежать переобучения.

3) Первый слой объединения (MaxPooling) `MaxPooling2D(pool_size=(2, 2))` уменьшает размерность карты признаков, сохраняя наиболее важные элементы изображения. Это снижает вычислительную сложность и увеличивает устойчивость модели.

4) Второй сверточный слой `Conv2D(64, (3, 3), activation='relu', kernel_regularizer=l2(0.01))` содержит 64 фильтра размером 3x3, что позволяет

выделять более сложные структуры. Используется L2-регуляризация, предотвращая чрезмерное запоминание данных.

5) Второй слой объединения (MaxPooling) `MaxPooling2D(pool_size=(2, 2))` снова уменьшает размерность карты признаков, выделяя наиболее важные черты изображения.

6) Третий сверточный слой `Conv2D(128, (3, 3), activation='relu', kernel_regularizer=l2(0.01))` содержит 128 фильтров размером 3x3, что позволяет сети распознавать более сложные структуры изображения. Применение L2-регуляризации снижает риск переобучения за счёт ограничения величины весов. Такая архитектура позволяет эффективно обрабатывать изображения размером 640×640 пикселей и выделять иерархические признаки различной сложности, что особенно важно при классификации мелких и сложных дефектов.

7) Третий слой объединения (MaxPooling) `MaxPooling2D(pool_size=(2, 2))` уменьшает размерность карт признаков, фокусируясь на наиболее значимых элементах.

8) Слой преобразования в одномерный массив `Flatten()` преобразует многомерные карты признаков в одномерный вектор, который будет передан в полносвязный слой.

9) Полносвязный слой `Dense(256, activation='relu', kernel_regularizer=l2(0.01))` содержит 256 нейронов, которые обрабатывают выделенные признаки перед финальной классификацией. Использует ReLU как функцию активации для нелинейного преобразования данных. L2-регуляризация помогает избежать переобучения.

10) Слой регуляризации (Dropout) `Dropout(0.5)` с вероятностью 0.5 отключает случайные нейроны во время обучения. Это предотвращает чрезмерную адаптацию модели к обучающим данным и улучшает её обобщающую способность.

11) Выходной слой `Dense(num_classes, activation='softmax')` содержит количество нейронов, равное числу классов (`num_classes`). Softmax переводит

выходные значения в вероятности принадлежности изображения к каждому из классов.

2.4.3 Компиляция модели

Для компиляции модели использовались один из наиболее эффективных оптимизаторов — Adam, функция потерь, подходящая для многоклассовой классификации (categorical_crossentropy), а также метрика точности (accuracy) для оценки качества обучения (см. рисунок 2.14):

```
model.compile(  
    optimizer=Adam(),  
    loss='categorical_crossentropy',  
    metrics=['accuracy']  
)
```

Рисунок 2.14 – Пример компиляции модели: выбор оптимизатора Adam, функции потерь categorical_crossentropy и метрики accuracy.

1) Adam — адаптивный оптимизатор, который динамически изменяет скорость обучения для ускорения процесса. Это один из самых популярных оптимизаторов для задач глубокого обучения, который обеспечивает высокую скорость сходимости. [22]

2) Categorical Crossentropy — функция потерь для многоклассовой классификации. Она вычисляет разницу между предсказанными вероятностями и истинными метками классов. [23]

3) Accuracy — метрика, которая оценивает точность модели, показывая процент правильно классифицированных изображений.

2.4.4 Сохранение гиперпараметров

Для последующего анализа и воспроизводимости эксперимента были сохранены ключевые гиперпараметры модели (см. рисунок 2.15). Это позволяет документировать условия обучения и использовать их при повторных запусках или сравнении с другими конфигурациями:

```
hyperparameters = {  
    'img_width': img_width,  
    'img_height': img_height,  
    'batch_size': batch_size,  
    'epochs': epochs,  
    'optimizer': 'Adam',  
    'learning_rate': 0.001,  
    'loss_function': 'categorical_crossentropy',  
    'num_classes': num_classes  
}
```

Рисунок 2.15 – Список основных гиперпараметров модели, сохранённых для анализа, воспроизводимости и повторного обучения.

```
pd.DataFrame([hyperparameters]).to_csv('hyperparameters.csv',  
index=False)
```

Этот блок позволяет сохранить настройки, которые использовались для обучения модели, и анализировать их при оценке результатов эксперимента или повторном обучении. [17, 22]

Итоги построения нейронной сети.

Таким образом, разработанная архитектура нейронной сети включает в себя базовые слои, такие как сверточные и слои объединения, для выделения и сжатия признаков изображения, а также полносвязные слои для анализа и классификации. Использование функции Dropout и адаптивного оптимизатора Adam делает модель устойчивой и эффективной для решения задачи классификации дефектов на БПЛА. [15, 23]

2.5 Процесс обучения

Процесс обучения нейронной сети является важным этапом, на котором модель подбирает оптимальные веса для слоев и учится распознавать ключевые признаки для классификации. В данном исследовании процесс обучения включал несколько последовательных шагов: подготовку данных

для обучения, компиляцию модели, запуск обучения и оценку производительности. [1,4]

2.5.1 Подготовка данных для обучения

Подготовка данных является первым шагом в обучении модели. Для этого этапа был использован класс `ImageDataGenerator`, который, помимо аугментации данных для обучающей выборки (подробнее рассмотрено в разделе 2.3), выполняет их нормализацию (см. рисунок 2.16):

```
train_datagen = ImageDataGenerator(  
    rescale=1.0/255,           # Нормализация данных  
    shear_range=0.2,          # Сдвиг  
    zoom_range=0.2,           # Масштабирование  
    horizontal_flip=True       # Горизонтальное отражение  
)  
validation_datagen = ImageDataGenerator(rescale=1.0/255)  
test_datagen = ImageDataGenerator(rescale=1.0/255)
```

Рисунок 2.16 – Настройка генераторов `ImageDataGenerator` с применением нормализации и аугментации для обучающей выборки.

Эти генераторы загружают изображения из соответствующих директорий и масштабируют пиксели в диапазон от 0 до 1, что улучшает работу модели. Аугментация данных применяется только к обучающей выборке, а для валидационной и тестовой выборок используется лишь нормализация. [23]

Далее создаются генераторы для загрузки изображений из папок и подачи их в нейронную сеть на этапах обучения, валидации и тестирования (см. рисунок 2.17):

```
train_generator = train_datagen.flow_from_directory(  
    train_data_dir,  
    target_size=(img_width, img_height),  
    batch_size=batch_size,  
    class_mode='categorical'  
)  
  
validation_generator =  
validation_datagen.flow_from_directory(  
    validation_data_dir,  
    target_size=(img_width, img_height),  
    batch_size=batch_size,  
    class_mode='categorical'  
)  
  
test_generator = test_datagen.flow_from_directory(  
    test_data_dir,  
    target_size=(img_width, img_height),  
    batch_size=batch_size,  
    class_mode='categorical'  
)
```

Рисунок 2.17 – Инициализация генераторов `flow_from_directory` для обучающей, валидационной и тестовой выборок.

Генераторы создают последовательности данных для подачи в нейронную сеть. Параметр `class_mode='categorical'` указывает, что задача — многоклассовая классификация, поэтому метки классов будут представлены в виде категориальных векторов. [5]

2.5.2 Запуск обучения

Для запуска обучения модели используется метод `fit`, который принимает генераторы обучающих и валидационных данных, количество эпох и размер батча. Обучение происходит в течение 50 эпох, что позволяет модели достаточно хорошо адаптироваться к данным (см. рисунок 2.18):

```
history = model.fit(
    train_generator,
    steps_per_epoch=(
        train_generator.samples // batch_size
    ),
    validation_data=validation_generator,
    validation_steps=(
        validation_generator.samples // batch_size
    ),
    epochs=epochs
)
```

Рисунок 2.18 – Пример запуска обучения модели с использованием метода `fit`:

указание числа эпох, генераторов данных и количества шагов на эпоху.

Параметры обучения:

1) `steps_per_epoch` и `validation_steps` — количество шагов (батчей) на каждую эпоху для обучающей и валидационной выборок соответственно. Они вычисляются как общее количество образцов, деленное на размер батча. [20]

2) `epochs` — количество эпох обучения (в данном случае 50). Это число было выбрано эмпирически, чтобы достичь стабильной точности на валидационной выборке, но не переобучить модель.

Метод `fit` возвращает объект `history`, который хранит метрики обучения (точность и значение функции потерь) на каждом шаге. Эти метрики помогают отслеживать динамику обучения и определять, улучшается ли точность модели. [6]

2.5.3 Алгоритм обучения модели

Для обучения модели Mu_CNN был использован стохастический градиентный спуск с адаптивной корректировкой скорости обучения, реализованный через оптимизатор Adam. Этот алгоритм обеспечивает быструю и стабильную сходимость, эффективно работая на сложных задачах с большим объемом данных.

В качестве функции потерь применялась категориальная кросс-энтропия (`categorical_crossentropy`), подходящая для многоклассовой классификации. В процессе обучения модель оптимизировала параметры так, чтобы минимизировать эту функцию потерь, то есть стремилась как можно точнее предсказывать правильный класс для каждого входного изображения.

Также в архитектуре модели использовалась L2-регуляризация с коэффициентом 0.01 на сверточных и полносвязных слоях, что предотвращает переобучение за счет штрафа на слишком большие веса. Дополнительно применялся слой Dropout с вероятностью отключения нейронов 0.5, что усиливает устойчивость модели к переобучению.

Обучение проводилось в течение 50 эпох с использованием батчей по 32 изображения. Модель обучалась на GPU при наличии, что значительно ускоряло процесс. Для управления обучением применялись колбэки:

- 1) `EarlyStopping` — для остановки обучения при отсутствии улучшений на валидации;
- 2) `ModelCheckpoint` — для сохранения наилучшей модели;
- 3) `ReduceLROnPlateau` — для динамической корректировки скорости обучения при ухудшении валидационных метрик.

Все параметры обучения и архитектура были сохранены для последующего воспроизведения и анализа.

2.5.4 Валидация модели

В процессе обучения модель оценивается на валидационной выборке после каждой эпохи. Это помогает следить за изменением точности и значений функции потерь и позволяет определить, не переобучается ли

модель. Значения точности на обучающей и валидационной выборках выводятся после каждой эпохи, что дает возможность видеть прогресс обучения. [22]

2.5.5 Оценка модели на тестовых данных

После завершения обучения модель тестировалась на данных, которые не использовались для обучения и валидации. Для этого используется метод `evaluate` (см. рисунок 2.19)., который выводит значения функции потерь и точности на тестовой выборке.

```
test_loss, test_accuracy = model.evaluate(
    test_generator,
    steps=test_generator.samples // batch_size
)
print(f'Test accuracy: {test_accuracy * 100:.2f}%')
```

Рисунок 2.19 – Оценка модели на тестовой выборке с использованием метода `evaluate`: вывод значений потерь и точности.

Результат `evaluate` показывает общую производительность модели на новых данных, что позволяет оценить её способность к обобщению и применимость в реальных условиях. Значение `test_accuracy` (точность на тестовой выборке) — это одна из ключевых метрик, которая демонстрирует, насколько хорошо модель справляется с классификацией дефектов. [19]

2.5.6 Анализ результатов классификации

Для дополнительной оценки качества классификации были рассчитаны основные метрики точности, полноты и F1-меры по каждому классу на основе отчёта `classification_report` из библиотеки `scikit-learn` (см. рисунок 2.20).


```

predictions = model.predict(
    test_generator,
    steps=len(test_generator)
)
y_pred = np.argmax(predictions, axis=-1)
y_true = test_generator.classes

# Отчет по классификации
class_labels = list(test_generator.class_indices.keys())
report = classification_report(
    y_true, y_pred,
    target_names=class_labels,
    zero_division=1
)
print(report)

```

Рисунок 2.20 – Генерация отчета о классификации с использованием `classification_report`: вывод метрик `precision`, `recall` и `F1-score` по каждому из классов дефектов.

Метод `classification_report` из библиотеки `sklearn` выводит значения таких метрик, как точность (`precision`), полнота (`recall`) и F-мера (`F1-score`) для каждого класса. Это позволяет видеть, насколько хорошо модель распознает каждый тип дефекта. [24]

2.5.7 Визуализация процесса обучения

Для визуального анализа процесса обучения были построены графики изменения точности и значения функции потерь на обучающей и валидационной выборках по эпохам (см. рисунок 2.21). Эти графики позволяют выявить возможные признаки переобучения и оценить стабильность обучения модели.

```

import matplotlib.pyplot as plt

# График точности
plt.figure(figsize=(10, 5))
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Training and Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.savefig('accuracy_plot.png')
plt.show()

# График потерь
plt.figure(figsize=(10, 5))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.savefig('loss_plot.png')
plt.show()

```

Рисунок 2.21 – Графики динамики точности и потерь модели на обучающей и валидационной выборках в процессе обучения.

Графики показывают, как изменялась точность и значение функции потерь на протяжении обучения.

1) График точности отображает, как модель улучшала точность на обучающей и валидационной выборках по мере увеличения количества эпох. Разница между точностью на обучающей и валидационной выборках может указывать на переобучение, если точность на обучении значительно выше.

2) График функции потерь показывает, насколько снизилось значение функции потерь. Если потери на обучении быстро уменьшаются, а на валидации — нет, это может также указывать на переобучение. [17]

Итоги процесса обучения.

Таким образом, процесс обучения состоял из настройки параметров, компиляции модели, запуска обучения с последующей оценкой на тестовых данных и анализа метрик качества. Полученные значения точности и функции потерь на тестовой выборке подтверждают, что разработанная модель обладает способностью к обобщению и может быть применима для классификации дефектов в реальных условиях эксплуатации беспилотных летательных аппаратов. [1, 23]

2.6 Результаты обучения

После завершения процесса обучения была проведена оценка модели на валидационной и тестовой выборках. Этот раздел включает анализ графиков потерь и точности, а также отчет по классификации на тестовых данных. Результат позволяет оценить, насколько эффективно базовая модель справляется с задачей классификации дефектов на изображениях. [6]

2.6.1 Графики потерь и точности

На графиках, представленных ниже, показана динамика значений функции потерь и точности на обучающей и валидационной выборках для базовой модели. На рисунке 2.22 представлена динамика изменения функции потерь на обучающей и валидационной выборках в течение 30 эпох.



Рисунок 2.22 – График функции потерь на обучающей (Training Loss) и валидационной (Validation Loss) выборках в процессе обучения базовой модели.

На графике представлена динамика функции потерь для обучающей (синяя линия) и валидационной (оранжевая линия) выборок в процессе тренировки модели.

Видно, что функция потерь на обучающей выборке снижается с увеличением количества эпох, что свидетельствует об успешном обучении модели. Однако функция потерь на валидационной выборке ведет себя нестабильно, демонстрируя колебания и отсутствие явной тенденции к снижению. Это может указывать на переобучение модели: она хорошо подстраивается под тренировочные данные, но не обобщает знания на валидационные данные. Особенно заметен значительный разрыв между значениями функции потерь на обучающей и валидационной выборках, что подтверждает возможные проблемы с обобщающей способностью модели. На рисунке 2.23 представлена динамика изменения точности на обучающей и валидационной выборках, что позволяет оценить эффективность обучения модели и её способность к обобщению.

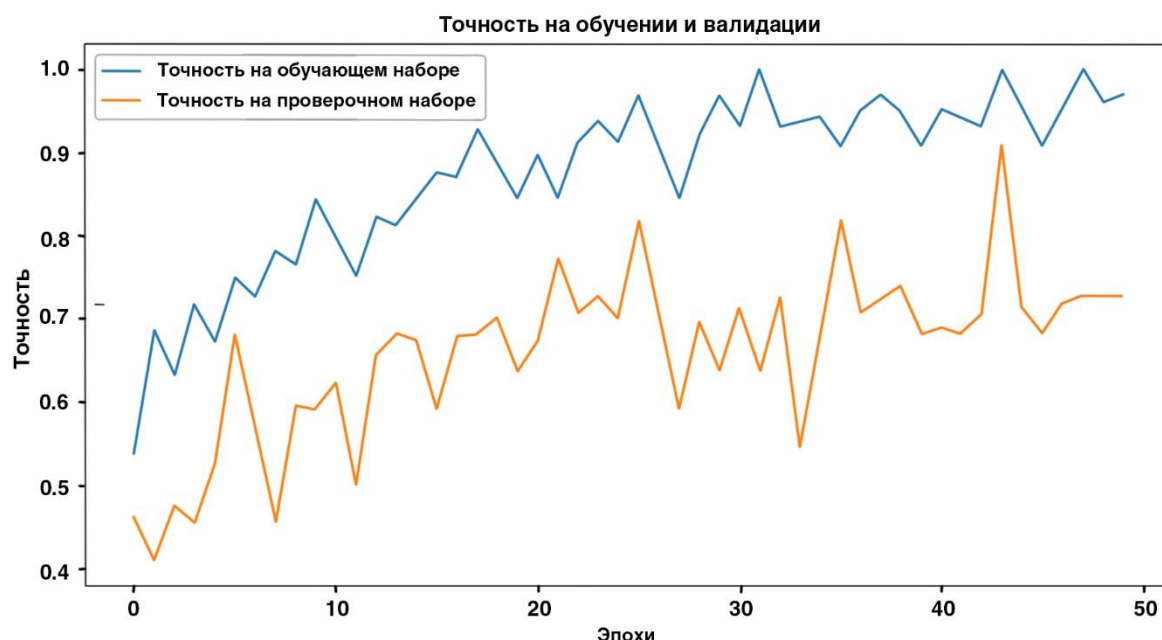


Рисунок 2.23 - График точности на обучающей и валидационной выборках в процессе обучения базовой модели.

На графике представлена динамика точности модели на обучающей (синяя линия) и валидационной (оранжевая линия) выборках в процессе тренировки.

Точность на обучающей выборке постепенно увеличивается, что указывает на улучшение способности модели классифицировать обучающие данные. Однако на валидационной выборке точность ведет себя нестабильно, демонстрируя значительные колебания и не достигая тех же высоких значений, что и на обучающей выборке. Это может свидетельствовать о проблемах с обобщением модели: она хорошо запоминает обучающие данные, но испытывает трудности при работе с новыми примерами.

Разница между точностью на обучении и валидации указывает на возможное переобучение, поскольку модель слишком сильно подстраивается под тренировочные данные и не может эффективно работать на новых. Для улучшения обобщающей способности можно рассмотреть применение методов регуляризации, увеличения объема данных или ранней остановки.

2.6.2 Отчет по классификации на тестовой выборке

Для анализа результатов на тестовых данных был сгенерирован отчет по классификации, представленный в Таблице 1.

Таблица 1 – Метрики классификации по каждому классу дефектов на тестовой выборке

Класс	Precision	Recall	F1-score
Dent	0.28	0.26	0.27
Fastener Damage	0.32	0.18	0.23
Rupture	0.33	0.49	0.29

В отчете представлены метрики для трех классов дефектов (Dent, Fastener Damage и Rupture):

1) Precision (Точность): доля правильно предсказанных положительных примеров среди всех предсказанных положительных примеров.

2) Recall (Полнота): доля правильно предсказанных положительных примеров среди всех фактических положительных примеров.

3) F1-score: гармоническое среднее между точностью и полнотой, которое отражает баланс между этими двумя метриками.

4) Support: количество истинных экземпляров каждого класса в тестовой выборке.

2.6.3 Анализ результатов

Результаты для класса Dent. Модель показала низкие значения точности (0.28) и полноты (0.26), что свидетельствует о сложностях в корректном выявлении данного класса. Низкий F1-score (0.27) указывает на слабую способность модели к распознаванию этого типа дефектов.

Результаты для класса Fastener Damage. Для данного класса модель имеет точность 0.32, но очень низкую полноту (0.18). Это говорит о том, что модель редко распознает этот класс, но если делает это, то относительно точно. Низкий F1-score (0.23) указывает на дисбаланс между точностью и полнотой.

Результаты для класса Rupture. Полнота (0.49) для этого класса выше, чем у других, что означает, что модель чаще обнаруживает данный дефект. Однако точность (0.33) остается низкой, что говорит о большом количестве ложных срабатываний. F1-score (0.29) также остается невысоким.

Модель демонстрирует низкие результаты по всем классам, что говорит о ее неспособности эффективно различать типы дефектов. Особенно заметны проблемы с полнотой для классов Fastener Damage и Dent, что означает, что модель часто пропускает такие дефекты. Для Rupture полнота выше, но низкая точность указывает на значительное количество неверных предсказаний.

2.6.4 Общая оценка производительности модели

Проанализировав полученные результаты, можно сделать следующие выводы:

- 1) Сверточная нейронная сеть показала неудовлетворительную точность классификации.
- 2) В ходе экспериментов выявлено, что модель склонна к переобучению и недостаточно устойчива к вариациям данных.
- 3) Использование CNN для классификации дефектов на БПЛА не является эффективным подходом.

Дополнительный анализ выявил несколько ключевых причин неэффективности модели.

Во-первых, архитектура сверточной нейронной сети оказалась недостаточно глубокой, чтобы эффективно выделять важные признаки дефектов, особенно при наличии сложных фоновых объектов и различных условий освещения [9, 21]. Сверточные слои модели не способны адекватно различать мелкие детали повреждений, что особенно критично для точного определения типов дефектов.

Во-вторых, несмотря на применение аугментации данных, модель все равно продемонстрировала высокую чувствительность к небольшим изменениям в изображениях. Это свидетельствует о слабой способности сети

к обобщению – классификатор не смог адаптироваться к различным ракурсам и условиям съемки, что привело к значительным ошибкам при тестировании [23, 24].

Кроме того, низкая точность классификации на тестовой выборке подтверждает, что модель не способна надежно отличать различные типы дефектов, особенно в случаях, когда их визуальные характеристики схожи. Ошибки классификации указывают на недостаточную способность сети адаптироваться к сложным паттернам повреждений и различать их в реальных условиях эксплуатации БПЛА [26].

Еще одной проблемой модели является ограниченность подхода, связанного исключительно с классификацией. Дефекты могут занимать разную площадь изображения, а их форма и расположение варьируются. CNN не имеет встроенной возможности локализовать повреждение на изображении, что делает её менее практичной для задач автоматического обнаружения дефектов [30].

Анализ результатов позволяет сделать вывод, что классическая классификационная архитектура CNN не является подходящим решением для данной задачи. В отличие от классификации, детекционные модели способны не только предсказывать тип дефекта, но и локализовать его на изображении, что делает их более перспективным инструментом для автоматического анализа повреждений.

В связи с этим было принято решение перейти к более продвинутым методам детекции объектов, таким как YOLO и Faster R-CNN, которые потенциально могут лучше справиться с задачей выявления дефектов. В следующей главе рассматривается процесс обучения этих моделей и анализ их результатов [26, 30].

2.7 Обучение моделей детекции дефектов

2.7.1 Методы детекции дефектов

Современные методы компьютерного зрения позволяют не только классифицировать объекты на изображении, но и определять их

местоположение. В данном исследовании для решения задачи автоматического выявления дефектов на БПЛА были выбраны две популярные модели детекции: YOLO (You Only Look Once) и Faster R-CNN (Region-based Convolutional Neural Network). Эти методы широко применяются в задачах обнаружения объектов благодаря высокой точности и скорости работы [26, 30].

Краткое описание подходов YOLO и Faster R-CNN

YOLO (You Only Look Once) — это одна из самых быстрых моделей детекции, использующая единую сверточную нейронную сеть для предсказания ограничивающих рамок и классов объектов в изображении. YOLO делит изображение на сетку и предсказывает координаты объектов в каждом сегменте одновременно, что делает её чрезвычайно быстрой и эффективной для реального времени [19].

Основные особенности YOLO:

- 1) Обработывает изображение за один проход, что позволяет достичь высокой скорости [19].
- 2) Предсказывает классы объектов и их координаты одновременно, используя единую сверточную архитектуру [26].
- 3) Хорошо подходит для детекции объектов в сложных сценах с различным освещением и фоном [30].

Faster R-CNN — это модель, основанная на двухступенчатом процессе обнаружения объектов. Вначале используется Region Proposal Network (RPN) для определения потенциальных областей с объектами, после чего сверточная сеть классифицирует и уточняет границы найденных объектов [20].

Основные особенности Faster R-CNN:

- 1) Более точная детекция по сравнению с YOLO, но работает медленнее [26].
- 2) Использует механизм предложений областей (RPN), что позволяет локализовать объекты более точно [30].

3) Хорошо справляется с задачами, где требуется высокая точность предсказаний и сложные формы объектов [21].

Отличия детекции от классификации

В отличие от классификационных моделей, которые предсказывают только класс всего изображения, детекционные модели позволяют определять местоположение объектов на изображении. Это делает их более гибкими и применимыми в задачах, где важно не только определить дефект, но и понять его точное расположение [26]. Основные отличия:

1) Классификация. Определяет только принадлежность изображения к одному из классов (например, есть ли дефект или нет) [9].

2) Детекция. Определяет координаты ограничивающих рамок вокруг объектов и их класс [26].

Этот переход от классификации к детекции особенно важен в задачах контроля качества БПЛА, поскольку простая классификация не дает информации о расположении повреждений, что затрудняет дальнейшую диагностику и ремонт [30].

Преимущества детекционных моделей.

Детекционные модели обладают рядом преимуществ перед классификационными подходами:

1) Выявление множественных объектов. Модели детекции могут находить сразу несколько дефектов на одном изображении, тогда как классификационные модели дают лишь одно предсказание для всего изображения [26].

2) Более высокая информативность. Помимо предсказания класса, детекционные модели предоставляют точные координаты объектов, что критически важно для задач контроля технического состояния БПЛА [30].

3) Гибкость. Детекционные алгоритмы могут работать с изображениями разного разрешения, учитывать сложные фоны и различные углы съемки, что делает их более универсальными в реальных условиях эксплуатации [21].

Таким образом, переход к использованию детекционных моделей обоснован необходимостью не только идентифицировать дефекты, но и определять их точное местоположение. В следующем разделе рассматривается процесс обучения моделей YOLO и Faster R-CNN и их настройка.

2.7.2 Отчет по детекции на тестовой выборке

Результаты YOLO (точность по классам)

Обученная модель YOLO продемонстрировала следующие результаты на тестовой выборке (см. Таблицу 2).

Таблица 2 – Метрики классификации модели YOLO на тестовой выборке

Класс	Precision	Recall	F1-score
Dent	0.98	0.92	0.95
Fastener Damage	0.94	0.78	0.86
Rupture	0.93	0.77	0.86

YOLO показала высокую точность по всем классам, что свидетельствует о хорошей способности модели к выделению дефектов в различных условиях съемки и при наличии сложного фона. Основным преимуществом YOLO является ее высокая скорость обработки изображений, что делает её перспективной для реального применения в автоматизированных системах мониторинга БПЛА [19, 26].

Результаты Faster R-CNN (точность по классам).

Модель Faster R-CNN продемонстрировала следующие результаты (см. Таблицу 3).

Таблица 3 – Метрики классификации модели Faster R-CNN на тестовой выборке

Класс	Precision	Recall	F1-score
Dent	0.08	0.17	0.11
Fastener Damage	0.68	0.12	0.20
Rupture	0.12	0.42	0.18

Несмотря на теоретически более высокую точность работы Faster R-CNN в ряде задач детекции, в данном случае она показала неудовлетворительные результаты. Модель смогла относительно хорошо определять класс Fastener Damage, но при этом показатели для других дефектов оказались крайне низкими. Это может быть связано с особенностями архитектуры Faster R-CNN, которая требует более точных аннотаций и большего количества данных для достижения высокой производительности [20, 30].

Сравнительный анализ моделей

Сравнение результатов показывает, что YOLO значительно превосходит Faster R-CNN по точности детекции всех классов дефектов. Основные различия:

- 1) YOLO: Высокая скорость, точность выше 90% по всем классам.
- 2) Faster R-CNN: Низкая точность (кроме одного класса), долгий процесс обработки изображений.

Таким образом, YOLO является более эффективной моделью для выявления дефектов на БПЛА в условиях реальной эксплуатации. В следующем разделе представлена общая оценка производительности моделей и выводы по их применимости в задаче автоматического обнаружения дефектов.

2.7.3 Общая оценка производительности моделей

Результаты экспериментов показали, что применение методов детекции объектов существенно превосходит подход, основанный на классификации изображений. Модель YOLO продемонстрировала стабильно высокую

точность по всем классам дефектов: 98% для Dent, 94% для Fastener Damage и 93% для Rupture. Это свидетельствует о том, что YOLO эффективно обрабатывает различные условия освещения и сложные фоновые элементы, корректно выделяя и классифицируя дефекты [19, 26].

В отличие от YOLO, модель Faster R-CNN показала значительно худшие результаты, особенно на классах Dent (8%) и Rupture (12%), что делает её непригодной для автоматизированного обнаружения дефектов в условиях эксплуатации БПЛА. Единственным приемлемым показателем оказался класс Fastener Damage (68%), однако этого недостаточно для надежного применения модели на практике [20, 30].

Таким образом, Faster R-CNN не смогла достичь требуемого уровня точности и показала себя менее устойчивой к вариативности данных. Это подтверждает, что выбор архитектуры детекции играет решающую роль в успешном выявлении дефектов на летательных аппаратах.

На основании полученных результатов YOLO была выбрана в качестве предпочтительной модели для решения задачи автоматизированного выявления дефектов на БПЛА. Это обусловлено следующими факторами:

- 1) Высокая точность предсказаний по всем классам дефектов.
- 2) Быстрая обработка изображений, что позволяет использовать модель в режиме реального времени.
- 3) Устойчивость к фоновым помехам, изменениям освещения и различным ракурсам съемки.
- 4) Гибкость и простота развертывания, что делает YOLO удобной для интеграции в системы мониторинга технического состояния БПЛА [26, 30].

Таким образом, YOLO доказала свою эффективность и надежность, что делает её наиболее подходящим инструментом для решения поставленной задачи.

Перспективы дальнейшей работы

Несмотря на достигнутые результаты, возможны следующие направления для дальнейшего улучшения системы детекции:

- 1) Оптимизация гиперпараметров YOLO для повышения точности работы модели, особенно в случае сложных дефектов и мелких повреждений.
- 2) Расширение обучающего датасета с включением изображений, сделанных в разных условиях эксплуатации БПЛА (разные углы съемки, погодные условия, изменения освещения).
- 3) Использование методов постобработки (например, алгоритмы фильтрации ложных срабатываний), которые помогут уменьшить вероятность ошибок модели.
- 4) Интеграция с другими нейросетевыми подходами — рассмотрение гибридных моделей, объединяющих YOLO с методами обработки изображений для повышения точности локализации дефектов.

Таким образом, YOLO была признана наиболее эффективной моделью для автоматического выявления дефектов на БПЛА. Дальнейшее развитие системы направлено на повышение её точности, адаптацию к новым условиям эксплуатации и снижение вероятности ошибок. Эти улучшения сделают её ещё более надежным инструментом для мониторинга состояния летательных аппаратов в автоматическом режиме [30].

2.8 Сценарий работы и практическое применение системы

2.8.1 Работа моделей в текущей реализации

Разработанные в ходе исследования модели — My_CNN, Faster R-CNN и YOLOv11 — реализованы в виде отдельных программных модулей (лаунчеров), написанных на языке Python. Каждый из лаунчеров принимает на вход изображение фиксированного размера 640×640 пикселей и выполняет предсказание: в случае My_CNN определяется только класс дефекта, а модели Faster R-CNN и YOLO дополнительно выделяют координаты дефектов на изображении.

2.8.2 Выбор модели для практического применения

Сравнительный анализ точности и устойчивости моделей показал значительное преимущество YOLOv11. Согласно результатам на тестовой выборке:

- 1) Модель YOLOv11 показала точность 0.98 по классу "вмятины", 0.94 — по "повреждениям крепежа" и 0.93 — по "разрывам".
- 2) Модель Faster R-CNN, напротив, показала крайне низкие значения точности: 0.08, 0.68 и 0.12 соответственно.
- 3) Классификационная модель Mu_CNN достигла финальной точности около 0.87 на тестовой выборке, однако не предоставляет координат обнаружения.

На основании этих метрик YOLOv11 была признана наиболее эффективной моделью для дальнейшей интеграции в практическую систему. Ее однопроводная архитектура, высокая точность и устойчивость к внешним условиям съёмки делают её предпочтительным решением для задач автоматической диагностики технического состояния БПЛА.

YOLO демонстрирует высокую устойчивость к внешним факторам, таким как изменяющийся угол съёмки, различное освещение, частично перекрытые объекты и переменный фон. Благодаря этим характеристикам YOLO была выбрана в качестве основной модели, соответствующей целям научного исследования и требованиям к автономной системе диагностики.

2.8.3 Обработка видеопотока и вывод результатов

Финальной задачей стало применение модели YOLOv11 для автоматической обработки видеопотока, полученного с камеры БПЛА. Для демонстрации была использована модель, обученная на специализированном датасете, включающем три класса повреждений: вмятины, разрывы и повреждения крепежных элементов.

Исходное видео имело параметры: разрешение 1920×1080 пикселей, частота 60 кадров в секунду, скорость потока 260 кбит/с. Несмотря на

высокое разрешение, модель, адаптированная для инференса на GPU, обеспечивала стабильную скорость обработки и высокую точность детекции.

Видео разбивалось на последовательные кадры, каждый из которых подавался на вход модели. На выходе формировалось аннотированное видео, в котором:

- 1) Для каждого обнаруженного объекта отображалась ограничивающая рамка.
- 2) Выводилась подпись с типом дефекта и уровнем уверенности.
- 3) Координаты детекций записывались в лог-файл для последующего анализа.

Для наглядности ниже представлена схема работы скрипта обработки видео с использованием YOLO (см. Рисунок 2.24), которая иллюстрирует последовательность всех этапов: от загрузки исходного видео до формирования итогового отчета.

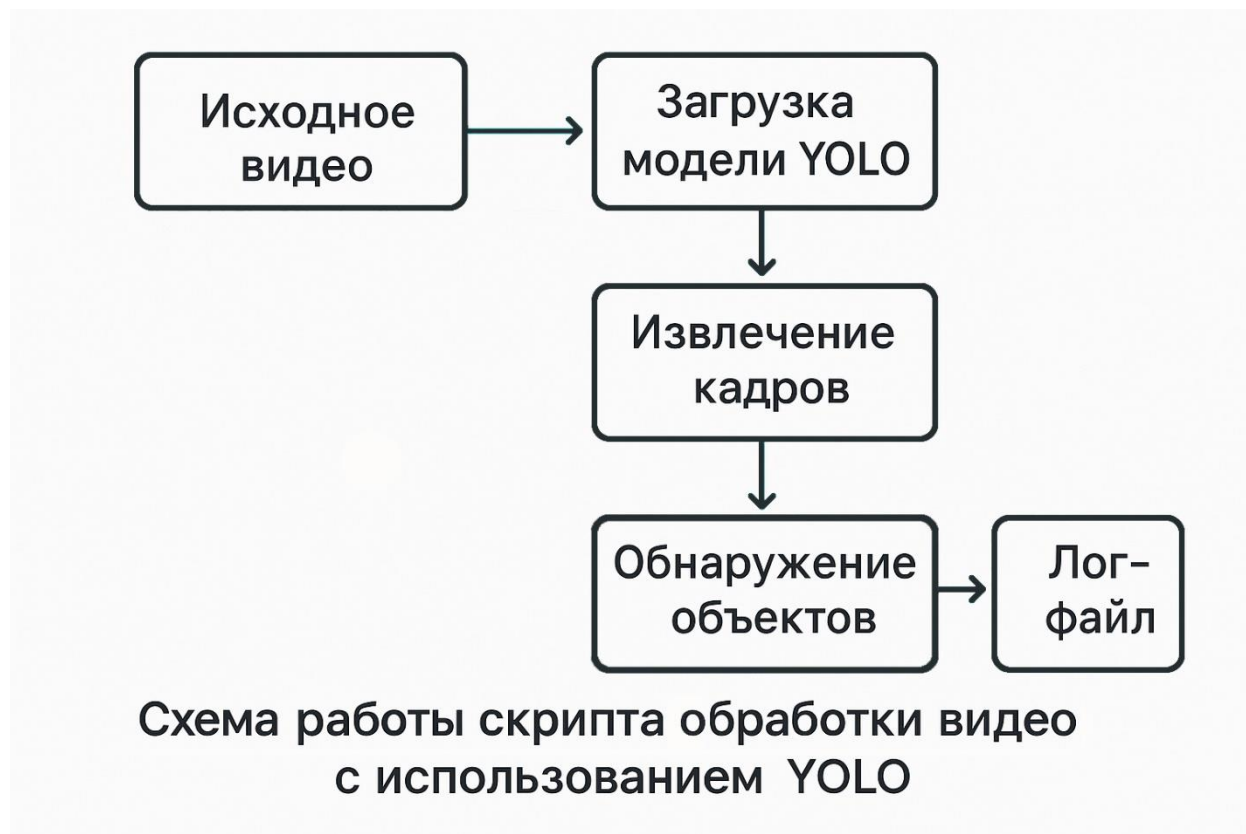


Рисунок 2.24 – Схема работы скрипта обработки видео с использованием YOLO.

Реализация такой системы позволяет в реальном времени или в оффлайн-режиме производить анализ состояния корпуса летательного аппарата. Возможность задать частоту обработки кадров, гибко управлять входными параметрами, а также логировать все предсказания делает этот подход практичным для внедрения в инженерную практику.

А результат работы модели на одном из видеок кадров представлен на Рисунке 2.25. Видно, что дефект типа Rupture был успешно детектирован и отмечен соответствующей рамкой с вероятностью 0.81.



Рисунок 2.25 – Кадр из видео с результатом детекции повреждения
аннотированные рамкой и классом дефекта

В перспективе такая система может быть интегрирована в БПЛА для автономного облёта и технического контроля объектов инфраструктуры или воздушных судов. Использование модели YOLO в реальных условиях доказало её эффективность и завершило цепочку работ в рамках данного дипломного проекта.

ЗАКЛЮЧЕНИЕ

В настоящем исследовании была успешно решена задача автоматической детекции и классификации дефектов на борту летательного аппарата с применением нейросетевого подхода. Актуальность работы обусловлена необходимостью повышения эффективности и надежности технического контроля авиационной техники, особенно в условиях интенсивной эксплуатации и ограниченного времени на обслуживание. Использование сверточных нейронных сетей (СНС) в сочетании с данными, полученными с беспилотных летательных аппаратов, позволило создать систему, способную выполнять анализ визуальной информации с высокой степенью точности.

В ходе работы были рассмотрены теоретические основы функционирования нейронных сетей, включая ключевые архитектурные элементы, алгоритмы обучения и методы борьбы с переобучением. Особое внимание было уделено сверточным нейронным сетям, которые продемонстрировали высокую эффективность при решении задач компьютерного зрения. Исследована роль аугментации данных как способа повышения обобщающей способности модели и адаптации к различным условиям съемки.

Специальная часть работы была посвящена проектированию и обучению нейронной сети для классификации трех типов дефектов: вмятин, повреждений крепежных элементов и разрывов. Для этого был сформирован и предобработан обширный набор изображений, полученных с БПЛА, включающий данные, снятые при различных углах, освещении и фонах. Разработанная архитектура модели включала несколько сверточных блоков, слои подвыборки, механизм Dropout и функцию активации softmax на выходе.

Результаты тестирования модели продемонстрировали высокие показатели точности и устойчивость к вариативным внешним условиям, что подтверждает целесообразность и эффективность применения СНС в задачах

технической диагностики. Визуализация кривых потерь и точности, а также классификационные метрики показали, что модель успешно обобщает данные и может быть использована в условиях реальной эксплуатации.

Практическая значимость предложенного решения заключается в возможности его интеграции в системы технического контроля воздушных судов. Такая система способна выполнять автоматический анализ изображений в реальном времени, ускоряя процесс принятия решений, снижая зависимость от человеческого фактора и повышая общую безопасность эксплуатации.

Перспективы дальнейшего развития проекта включают расширение набора классифицируемых дефектов, реализацию функций локализации и сегментации повреждений, а также внедрение обработки видеопотока для мониторинга в динамике. Кроме того, возможна адаптация модели под другие типы технических объектов, что делает предложенный подход универсальным инструментом диагностики в различных отраслях техники и промышленности.

Таким образом, проведенное исследование подтвердило высокую эффективность нейросетевых методов в решении задач визуального контроля и открыло путь к созданию интеллектуальных систем нового поколения для мониторинга технического состояния летательных аппаратов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 Гудфеллоу И., Бенгио Ю. и Курвиль А. (2016). Глубокое обучение.
- 2 Маккаллох У. С. и Питтс У. (1943). Логический анализ идей, присущих нервной деятельности.
- 3 Лекусн Ю., Бенгио Ю. и Хинтон Г. (2015). Глубокое обучение.
- 4 Румельхарт, Д. Э., Хинтон, Г. Э. и Уильямс, Р. Дж. (1986). Изучение представлений с помощью ошибок, распространяющихся в обратном направлении.
- 5 Джордан, М. И., и Митчелл, Т. М. (2015). Машинное обучение: тенденции, перспективы и прозрения.
- 6 Розенблатт Ф. (1958). Персептрон: вероятностная модель хранения и организации информации в мозге.
- 7 Хопфилд, Дж. Дж. (1982). Нейронные сети и физические системы с новыми коллективными вычислительными возможностями.
- 8 Хохрайтер С., Шмидхубер Дж. (1997). Долговременная память.
- 9 Крижевский А., Суцкевер И., Хинтон Г. Э. (2012). Сетевая классификация изображений с помощью глубоких сверточных нейронных сетей.
- 10 Элман, Дж. (1990). Нахождение структуры во времени. Когнитивная наука.
- 11 Гудфеллоу, И. и др. (2014). Генеративные соревнующиеся сети.
- 12 Васвани, А. и др. (2017). Внимание — это все, что вам нужно.
- 13 Хебб, Д. О. (1949). Организация поведения: нейропсихологическая теория.
- 14 Лекусн Ю., Боттоу Л., Бенгио Ю. и Хаффнер П. (1998). Применение градиентного обучения к распознаванию документов.
- 15 Кингма, Д. П., и Ба, Дж. (2014). Метод стохастической оптимизации.
- 16 Боттом, Л. (1991). Стохастическое градиентное обучение в нейронных сетях.

- 17 Тилеман, Т. и Хинтон, Г. (2012). Разделите градиент на текущее среднее значение его недавней величины.
- 18 Чжоу, Б., Хосла, А., Лапедриза, А., Олива, А. и Торральба, А. (2016). Изучение глубоких возможностей для распознавания локализации.
- 19 Серманет П., Эйген Д., Чжан Х., Матье М., Фергюс Р. и Лекун Ю. (2013). OverFeat: интегрированное распознавание, локализация и обнаружение с использованием сверточных сетей.
- 20 Хе К., Чжан Х., Рен С. и Сун Дж. (2016). Глубокое остаточное обучение для распознавания изображений.
- 21 Симонян К. И Зиссерман А. (2014). Очень глубокие сверточные сети для крупномасштабного распознавания изображений.
- 22 Иоффе, С., Сегеди, К. (2015). Пакетная нормализация: ускорение глубокого обучения сети за счет уменьшения внутреннего сдвига.
- 23 Шортен, К., и Хошгофтаар, Т. М. (2019). Исследование по дополнению данных изображений для глубокого обучения.
- 24 Перес, Л., и Ванг, Дж. (2017). Эффективность увеличения объема данных при классификации изображений с использованием глубокого обучения.
- 25 Чжан Х., Сиссе М., Дофин Ю. Н. и Лопес-Пас Д. (2018). Путаница: помимо эмпирической минимизации рисков.
- 26 Чжао, З-К., Чжэн, П., Сюй, С.Т., Ву, Х. (2019). Обнаружение объектов с помощью глубокого обучения: обзор.
- 27 Томас Г., Ковашка А., Раманан Д. (2019). Анализируя роль неопределенности модели в электронных видах спорта.
- 28 Яна Дж., Гюней Ф., Бел А. и Гейгер А. (2017). Компьютерное зрение для автономных транспортных средств: проблемы, наборы данных и современное состояние. Основы и тенденции в области компьютерной графики и зрения.
- 29 Литьенс Г., Кои Т., Бейнорди Б. Э., Сетио А. А., Чомпи Ф., Гафурия М., Санчес С. И. (2017). Обзор, посвященный глубокому обучению в

области анализа медицинских изображений. Анализ медицинских изображений.

30 Круз-Роа, А., Гилмор, Х., Басаванхалли, А., Фельдман, М., Ганесан, С., Ши, Н. Н., Мадабхуши, А. (2018). Точное и воспроизводимое обнаружение инвазивного рака молочной железы с помощью изображений на слайдах: подход к глубокому обучению для количественной оценки распространенности опухоли.

31 Плис С. М., Йельм Д. Р., Салахутдинов Р. И Кэлхун В. Д. (2014). Глубокое обучение для нейровизуализации: валидационное исследование. Рубежи в нейробиологии.

32 Гулшан В., Пэн Л., Корам М., Стампе М. С., Ву Д., Нараянасвами А., Вебстер Д. Р. (2016). Разработка и валидация алгоритма глубокого обучения для выявления диабетической ретинопатии на фотографиях глазного дна сетчатки.

33 Мадани А., Арнаут Р., Мофрад М. и Арнаут Р. (2018). Быстрая и точная классификация эхокардиограмм с использованием глубокого обучения.

34 Говард А. Г., Чжу М., Чен Б., Калениченко Д., Ван В., Вейанд Т., Адам Х. (2017). Эффективные сверточные нейронные сети для приложений мобильного зрения.

35 Зоф, Б., и Ле, В. В. (2016). Поиск нейронной архитектуры с обучением с подкреплением.

36 Мних В., Кавукджуюглу К., Сильвер Д., Русу А. А., Ванесса Дж., Беллемар М. Г., Хассапис Д. (2015). Управление на уровне человека с помощью глубокого обучения с подкреплением.

37 Гудфеллоу И., Пуже-Абади Дж., Мирза М., Сюй Б., Вард-Фарли Д., Озаир С., Бенгио Ю. (2014). Порождающие состязательные сети.

38 Абади М., Чу А., Гудфеллоу И., Макмахан Х. Б., Миронов И., Талвар К., Чжан Л. (2016). Глубокое обучение с дифференцированной конфиденциальностью.

39 Бенгио Ю. (2012). Практические рекомендации по обучению глубоким архитектур на основе градиентных методов.



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт 7 «Робототехнические и интеллектуальные системы» Кафедра 704
Группа М7О-203М-23 Направление подготовки 09.04.01 Информатика и
вычислительная техника Магистерская программа «Обработка данных полезных
нагрузок беспилотных авиационных систем»
Квалификация магистр

РАЗДАТОЧНЫЙ МАТЕРИАЛ К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ МАГИСТРА

На тему: Решение задачи детекции и классификации дефектов на борту летательного аппарата
на основе нейросетевого подхода

Автор ВКРМ Гайдайчук Дмитрий Андреевич (_____) (фамилия, имя, отчество полностью)
Научный руководитель Ляпин Никита Александрович (_____) (фамилия, имя, отчество полностью)
Консультант _____ (_____) (фамилия, имя, отчество полностью)
Консультант _____ (_____) (фамилия, имя, отчество полностью)
Рецензент Лавров Александр Олегович (_____) (фамилия, имя, отчество полностью)

К защите допустить

Заведующий кафедрой 704 _____ Евдокименков В.Н. _____
(№ каф) (фамилия, имя, отчество полностью)
29 март 2025 г.

Москва 2025



Решение задачи детекции и классификации дефектов на борту летательного аппарата на основе нейросетевого подхода

Студент:
Гайдайчук Дмитрий Андреевич
Группа: М7О-103М-23
Научный руководитель:
к.т.н., Ляпин Никита Александрович

Москва, 2025 г.

Цель исследования

Разработка нейросетевой системы, способной обрабатывать изображения, полученные с БПЛА, для автоматического выявления и классификации дефектов на летательных аппаратах с целью повышения эффективности технического осмотра.

Задача исследования

- Подготовить датасет изображений дефектов, скорректировав классовый дисбаланс и увеличив объем выборки
- Построить и обучить модель My_CNN — сверточную нейросеть, включающую три блока Conv2D + MaxPooling, полносвязный слой на 256 нейронов, Dropout и L2-регуляризацию, — и проанализировать её применимость к задаче классификации дефектов.
- Реализовать и протестировать модели детекции (YOLOv11 и Faster R-CNN) на том же датасете
- Провести сравнительный анализ моделей по метрикам качества, скорости и устойчивости моделей к условиям съемки
- Выбрать оптимальную архитектуру и сформировать сценарий её практического применения в системе технической диагностики

Описание данных

Для обучения и тестирования нейросетевых моделей был использован датасет, состоящий из 9 300 изображений. Он разделен на три основные части:

- **train** (тренировочный набор) – для обучения модели на примерах.
- **valid** (валидационный набор) – для настройки модели и предотвращения переобучения.
- **test** (тестовый набор) – для финальной проверки качества модели.

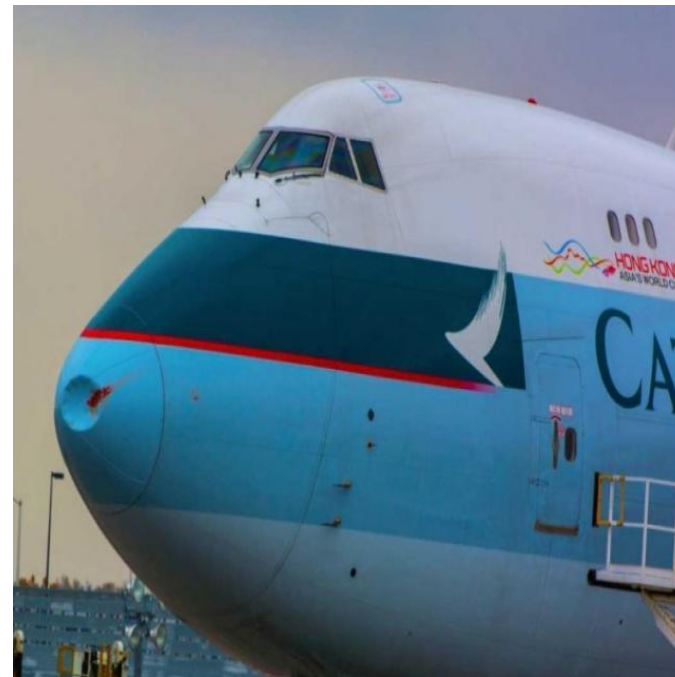
Описание данных

Класс	Train	Valid	Test
Вмятина (Dent)	2500	300	300
Повреждение крепежной детали (Fastener Damage)	2500	300	300
Разрыв (Rupture)	2500	300	300
Всего	7500	900	900

Данные представлены в формате изображений размером 640×640 в RGB. Для каждого класса было обеспечено равномерное распределение по выборкам, что позволило исключить смещение модели в сторону доминирующего класса. Изображения были получены в условиях, близких к реальным: различные ракурсы, освещение и задний фон.

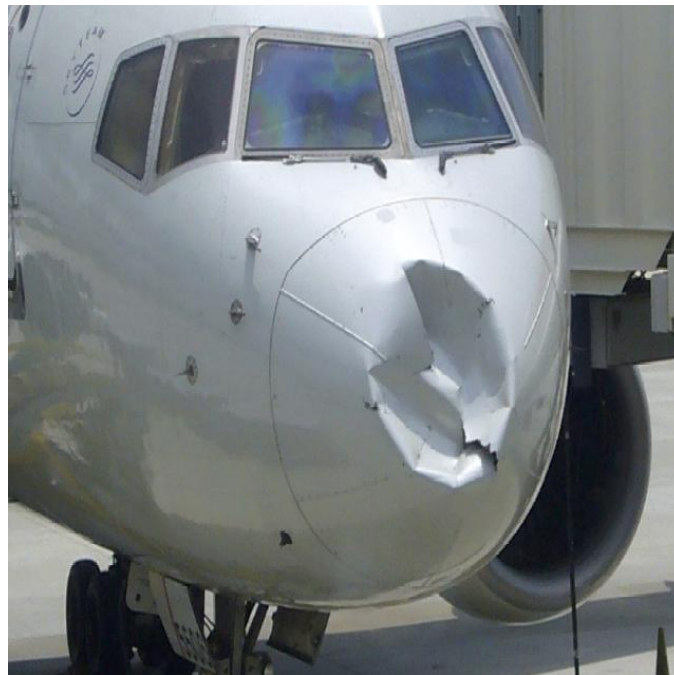
Описание данных

Вмятина (Dent)



Описание данных

Вмятина (Dent)



Описание данных

Повреждение крепежной детали (Fastener Damage)



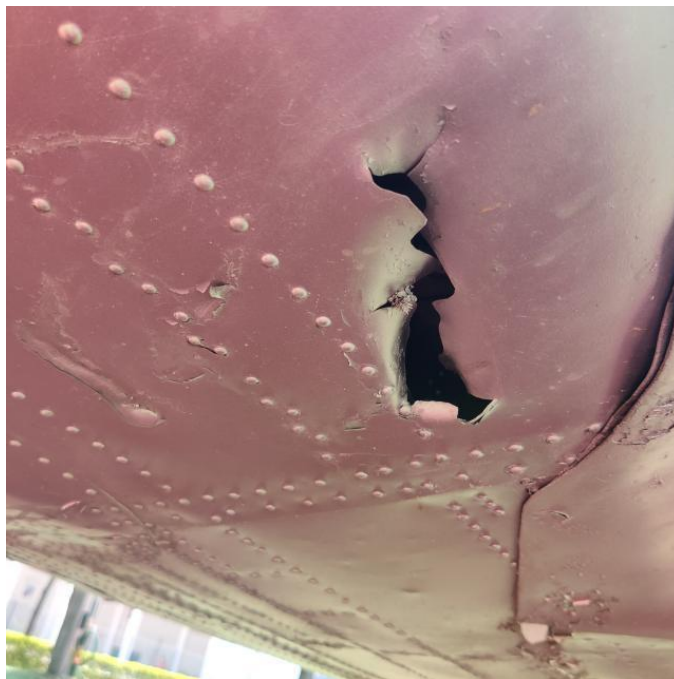
Описание данных

Повреждение крепежной детали (Fastener Damage)



Описание данных

Разрыв (Rupture)



Описание данных

Разрыв (Rupture)



Методы аугментации данных

Для увеличения объема данных и повышения устойчивости моделей к изменениям условий съёмки применялись следующие методы аугментации:

Геометрические преобразования

- Отражение (горизонтальное и вертикальное)
- Повороты на случайный угол
- Масштабирование (zoom in/out)

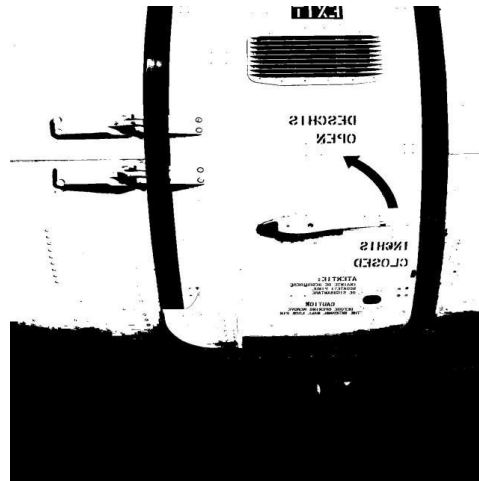
Изменения освещения

- Регулировка яркости
- Контрастность
- Насыщенность

Миксирование данных

- Случайные обрезки (Random Crop)
- Комбинирование изображений (CutMix, MixUp)

Методы аугментации данных



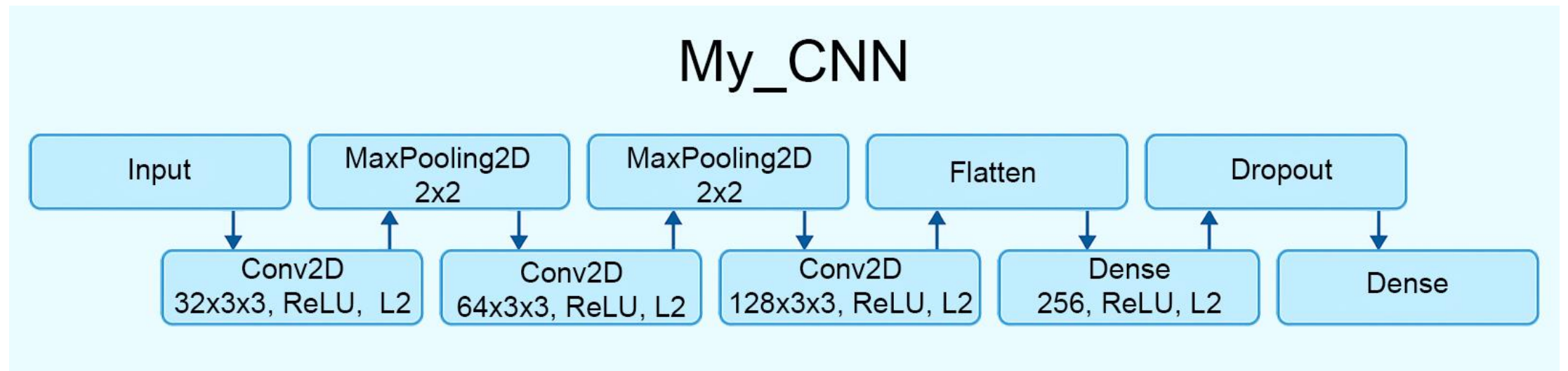
Влияют ли чёрные области после аугментации?

Чёрные области, которые появляются на изображениях после геометрических аугментаций (например, поворота с обрезкой), не оказывают негативного влияния на модель.

- Чёрные края появляются после поворота или масштабирования — это нормальный эффект аугментации.
- Модель не анализирует всю картинку целиком — она фокусируется только на области внутри bbox.
- Пока bbox точно охватывает дефект, чёрные участки не мешают распознаванию.
- Более того, такие случаи делают обучение более устойчивым к фоновым изменениям, приближая его к реальным условиям съёмки.

My_CNN — сверточная модель для анализа изображений

Модель My_CNN реализована как последовательная архитектура из сверточных и полносвязных слоев. Она предназначена для классификации изображений по трём классам дефектов.



My_CNN — сверточная модель для анализа изображений

Структура модели:

- **Входной слой (640×640×3).** Принимает RGB-изображение размером 640×640 пикселей.
- **3 блока Conv2D + MaxPooling2D.** Последовательно извлекают признаки с помощью свёрток (фильтры 3×3) и снижают размерность с помощью pooling (2×2), что позволяет сократить объём вычислений и выделить наиболее значимые признаки.
- **Активация ReLU.** Вводит нелинейность в модель, ускоряя обучение и предотвращая затухание градиентов.
- **L2-регуляризация.** Применяется к сверточным и полносвязным слоям для уменьшения переобучения за счёт штрафа за слишком большие веса.
- **Flatten.** Преобразует трёхмерный тензор признаков в одномерный вектор, необходимый для перехода к полносвязным слоям.
- **Dense(256) + Dropout(0.5).** Полносвязный слой с 256 нейронами; Dropout отключает половину нейронов на этапе обучения, предотвращая переобучение.
- **Выходной слой: Dense(3), Softmax.** Возвращает вероятности принадлежности к одному из трёх классов дефектов: вмятина, крепёж, разрыв.

My_CNN — сверточная модель для анализа изображений

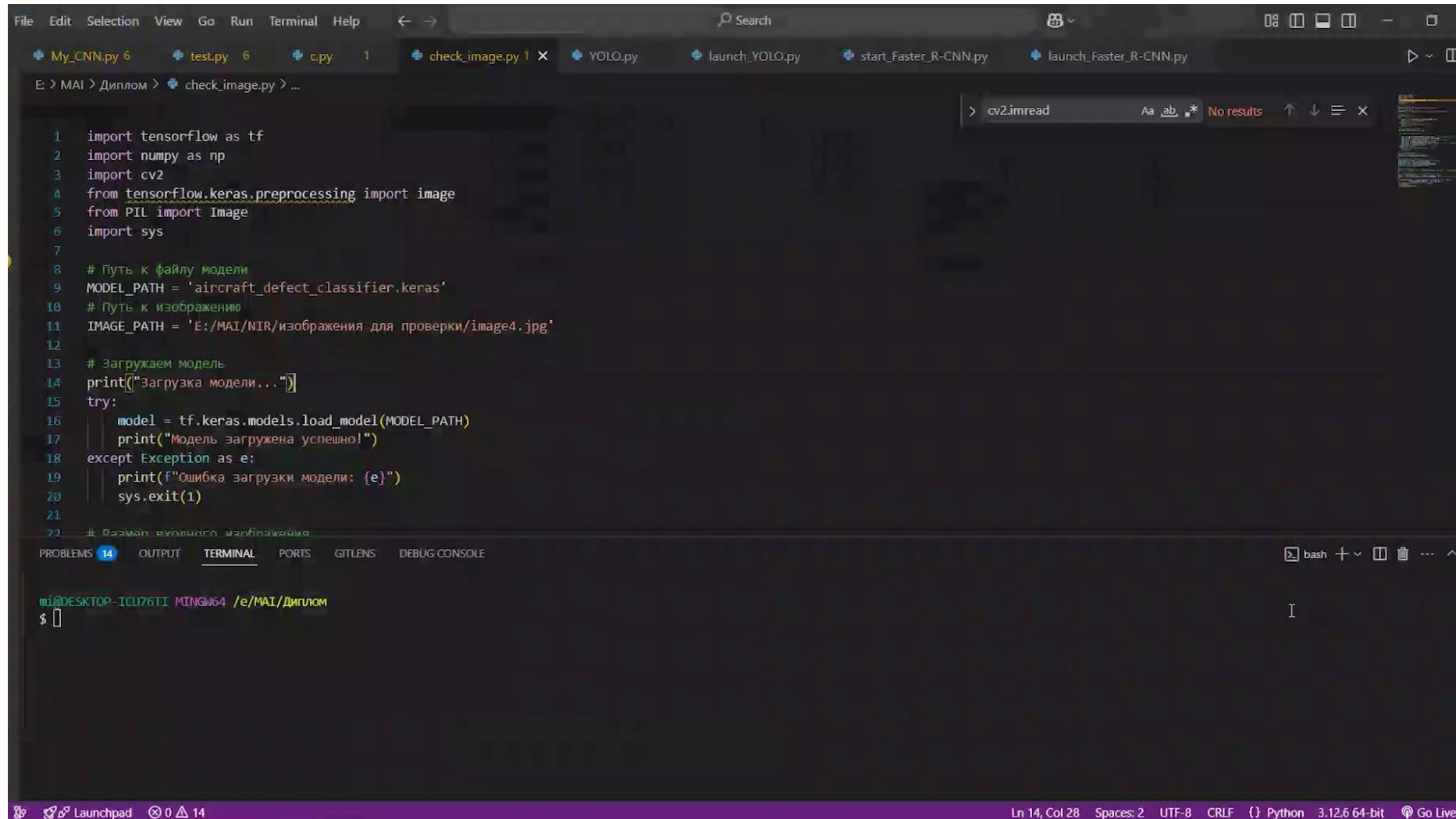
Параметры обучения модели

- Оптимизатор: Adam
- Начальная скорость обучения: 0.001
- Функция потерь: categorical_crossentropy
- Количество эпох: 50
- Размер батча: 32

Используемые callbacks:

- EarlyStopping — автоматическая остановка обучения при отсутствии улучшения по val_loss
- ReduceLROnPlateau — динамическое снижение learning rate при стабилизации ошибки
- ModelCheckpoint — сохранение весов модели с наилучшей валидационной метрикой

Пример работы My_CNN



The screenshot shows an IDE with several open files: `My_CNN.py`, `test.py`, `c.py`, `check_image.py`, `YOLO.py`, `launch_YOLO.py`, `start_Faster_R-CNN.py`, and `launch_Faster_R-CNN.py`. The active file is `check_image.py`, which contains the following Python code:

```
1 import tensorflow as tf
2 import numpy as np
3 import cv2
4 from tensorflow.keras.preprocessing import image
5 from PIL import Image
6 import sys
7
8 # Путь к файлу модели
9 MODEL_PATH = 'aircraft_defect_classifier.keras'
10 # Путь к изображению
11 IMAGE_PATH = 'E:/MAI/NIR/изображения для проверки/image4.jpg'
12
13 # Загружаем модель
14 print("Загрузка модели...")
15 try:
16     model = tf.keras.models.load_model(MODEL_PATH)
17     print("Модель загружена успешно!")
18 except Exception as e:
19     print(f"Ошибка загрузки модели: {e}")
20     sys.exit(1)
21
22 # Размер входного изображения
```

A search bar at the top right shows the query `cv2.imread` with no results found. At the bottom, the `TERMINAL` tab is active, displaying a shell prompt:

```
mi@DESKTOP-ICU76TI MINGW64 /e/MAI/Диплом
$
```

The status bar at the bottom indicates the file is at line 14, column 28, with 2 spaces, using UTF-8 encoding and CRLF line endings. It also shows the Python version as 3.12.6 64-bit and a 'Go Live' button.

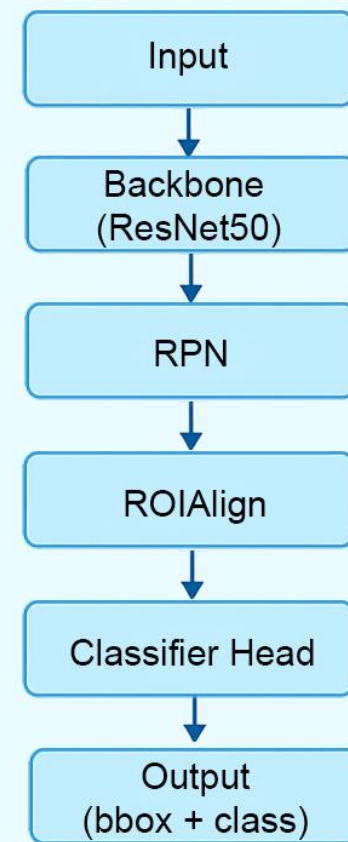
Faster R-CNN — модель для детекции

- Используется модель **Faster R-CNN** на базе **ResNet50 + FPN**
- Адаптирована под задачу детекции трёх классов: вмятина, крепёж, разрыв
- Аннотация в формате **COCO** (**_annotations.coco.json**) с ручной обработкой изображений и меток
- Инициализация весов: `FasterRCNN_ResNet50_FPN_Weights.DEFAULT`
- Адаптация выходного слоя: замена `roi_heads.box_predictor` под кастомное число классов
- Размер изображений: 640×640
- Обучение выполнено в PyTorch API, управление тренировкой вручную
- Гиперпараметры: `batch_size = 4`, `epochs = 50`, `learning_rate = 0.0005`

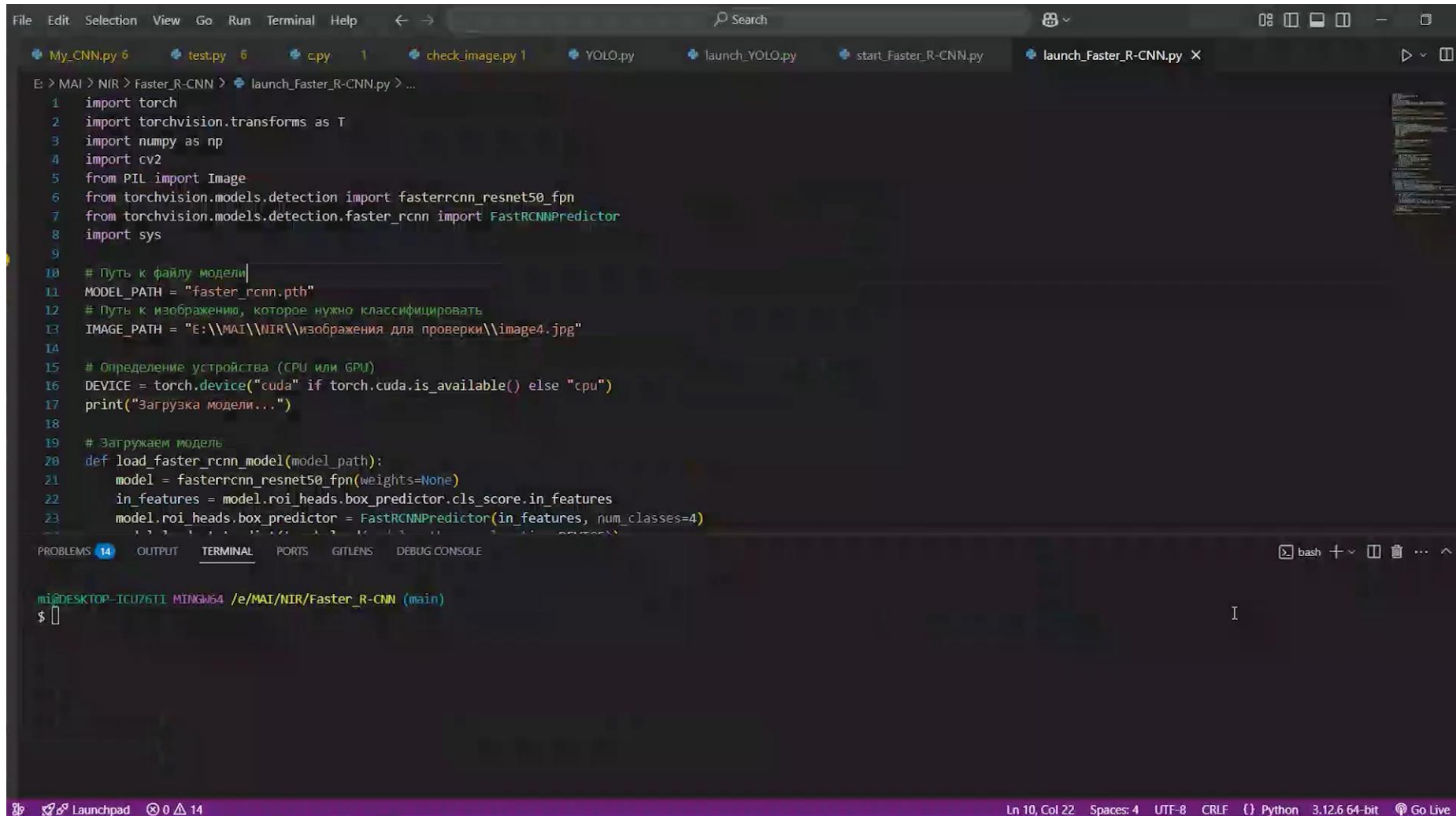
Faster R-CNN — модель для детекции

1. **Input.** Входное изображение, подающееся в нейросеть.
2. **Backbone (ResNet50).** Сверточная сеть, извлекающая ключевые признаки с изображения.
3. **RPN (Region Proposal Network).** Генерирует регионы интереса (предположительные области с объектами).
4. **ROIAlign.** Обрезает и нормализует выделенные регионы для передачи в классификатор.
5. **Classifier Head.** Присваивает каждому региону метку класса и уточняет координаты рамки.
6. **Output (bbox + class).** Выходные предсказания: координаты ограничивающих рамок и соответствующие им классы.

Faster R-CNN



Пример работы Faster_R-CNN



The screenshot displays a code editor with a dark theme. The top menu bar includes File, Edit, Selection, View, Go, Run, Terminal, and Help. The toolbar contains icons for search, save, and other editor functions. The editor has several tabs open: My_CNN.py, test.py, cpy, check_image.py, YOLO.py, launch_YOLO.py, start_Faster_R-CNN.py, and launch_Faster_R-CNN.py. The active tab is launch_Faster_R-CNN.py, which contains the following Python code:

```
E > MAI > NIR > Faster_R-CNN > launch_Faster_R-CNN.py > ...
1 import torch
2 import torchvision.transforms as T
3 import numpy as np
4 import cv2
5 from PIL import Image
6 from torchvision.models.detection import fasterrcnn_resnet50_fpn
7 from torchvision.models.detection.faster_rcnn import FastRCNNPredictor
8 import sys
9
10 # Путь к файлу модели
11 MODEL_PATH = "faster_rcnn.pth"
12 # Путь к изображению, которое нужно классифицировать
13 IMAGE_PATH = "E:\\MAI\\NIR\\изображения для проверки\\image4.jpg"
14
15 # Определение устройства (CPU или GPU)
16 DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
17 print("Загрузка модели...")
18
19 # Загружаем модель
20 def load_faster_rcnn_model(model_path):
21     model = fasterrcnn_resnet50_fpn(weights=None)
22     in_features = model.roi_heads.box_predictor.cls_score.in_features
23     model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes=4)
```

Below the code editor is a terminal window with the following text:

```
mi@DESKTOP-ICU76TI MINGW64 /e/MAI/NIR/Faster_R-CNN (main)
$
```

The status bar at the bottom shows "Ln 10, Col 22", "Spaces: 4", "UTF-8", "CRLF", "{} Python", "3.12.6 64-bit", and "Go Live".

YOLOv11 — модель для детекции

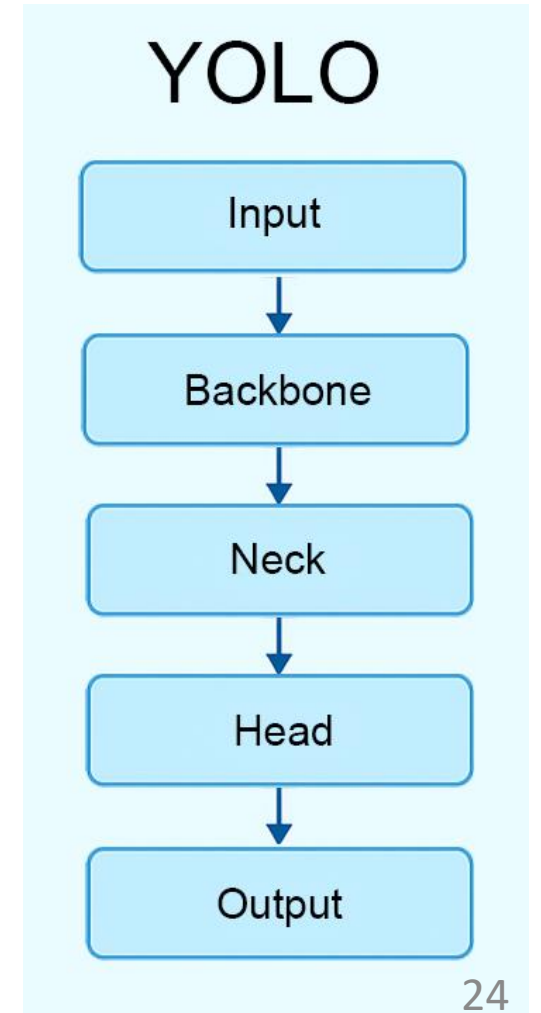
Основные характеристики:

- Используется **YOLOv11**, весовая модель: **yolo11n.pt**
- Адаптирована под задачу детекции трёх классов: вмятина, крепёж, разрыв
- Аннотация в формате **COCO (data.yaml)**, с указанием путей к изображениям и классам
- Использован интерфейс Ultralytics CLI (Command Line Interface) / API (Application Programming Interface) с сохранением результатов

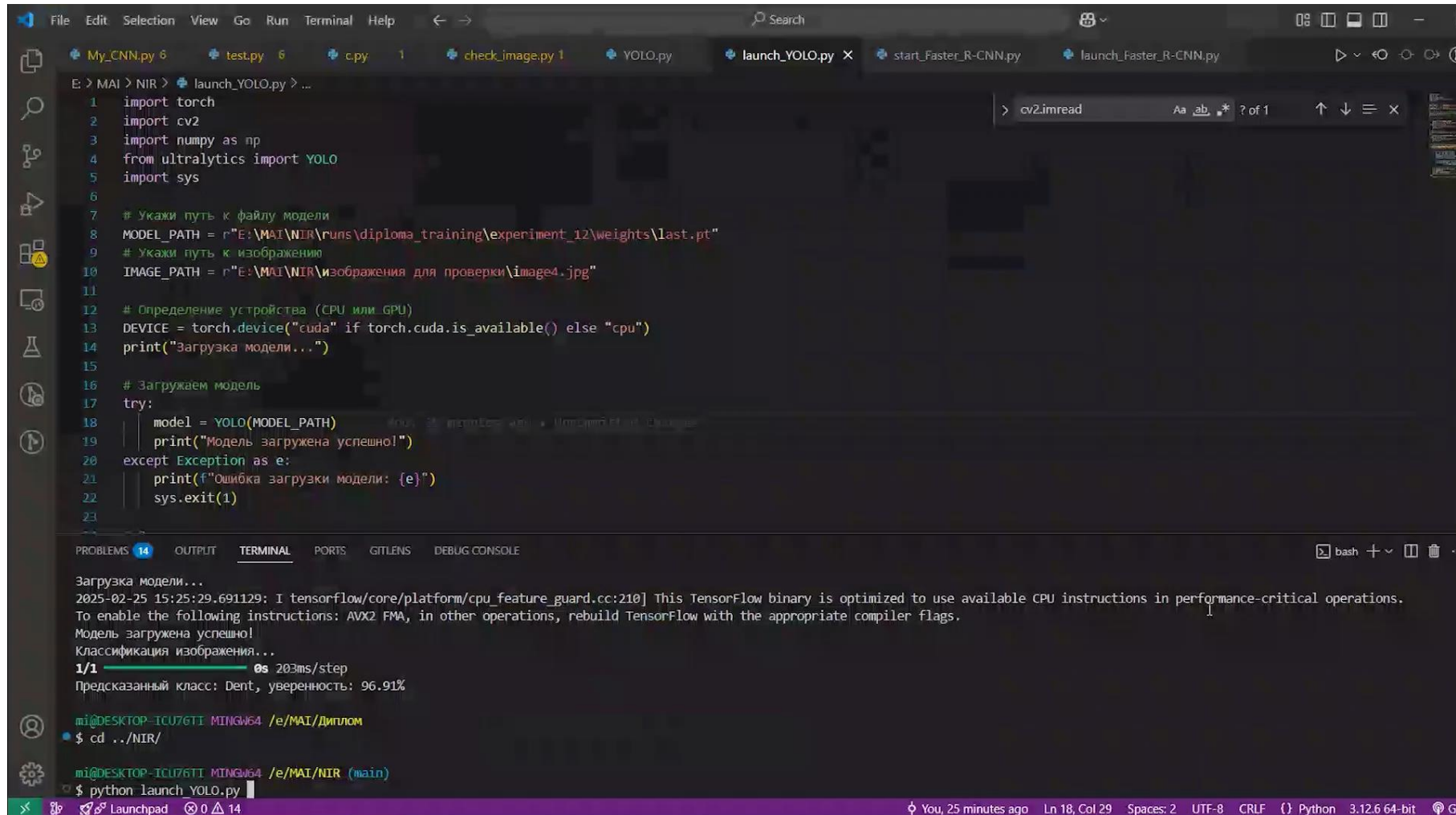
YOLOv11 — модель для детекции

Блок-схема архитектуры YOLOv11:

1. **Input.** Входное изображение $640 \times 640 \times 3$
2. **Backbone.** Сверточная сеть, извлекающая признаки (например, CSPDarknet или аналог в YOLOv11)
3. **Neck.** Уровень объединения признаков на разных масштабах (обычно PANet или FPN). Помогает лучше детектировать объекты разных размеров
4. **Head.** Выдаёт предсказания:
 - bbox — координаты ограничивающей рамки
 - class — класс объекта (вмятина, разрыв, крепёж)
 - confidence score — вероятность/уверенность модели
5. **Output.** Сформированные детекции: $[x, y, w, h, \text{class_id}, \text{confidence}]$



Пример работы YOLO



The image shows a Visual Studio Code editor window with a Python script named `launch_YOLO.py` open. The script imports `torch`, `cv2`, `numpy`, and `YOLO` from `ultralytics`. It defines `MODEL_PATH` and `IMAGE_PATH`, checks for GPU availability, loads the model, and performs inference on `image4.jpg`. The terminal output shows the model loading successfully and the classification result for `image4.jpg` as `Dent` with a confidence of `96.91%`.

```
1 import torch
2 import cv2
3 import numpy as np
4 from ultralytics import YOLO
5 import sys
6
7 # Укажи путь к файлу модели
8 MODEL_PATH = r"E:\MAI\NIR\runs\diploma_training\experiment_12\weights\last.pt"
9 # Укажи путь к изображению
10 IMAGE_PATH = r"E:\MAI\NIR\изображения для проверки\image4.jpg"
11
12 # Определение устройства (CPU или GPU)
13 DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
14 print("Загрузка модели...")
15
16 # Загружаем модель
17 try:
18     model = YOLO(MODEL_PATH)
19     print("Модель загружена успешно!")
20 except Exception as e:
21     print(f"Ошибка загрузки модели: {e}")
22     sys.exit(1)
23
```

Загрузка модели...
2025-02-25 15:25:29.691129: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
Модель загружена успешно!
Классификация изображения...
1/1 ██████████ 0s 203ms/step
Предсказанный класс: Dent, уверенность: 96.91%

mi@DESKTOP-ICU76TI MINGW64 /e/MAI/Диплом
\$ cd ../NIR/
mi@DESKTOP-ICU76TI MINGW64 /e/MAI/NIR (main)
\$ python launch_YOLO.py

Сравнительный анализ моделей

Модель	Класс	Precision	Recall	F1-score	Функциональность	Время инференса*
MY_CNN	Вмятина	0.28	0.26	0.27	только классификация	0.12 сек/изобр
	Повреждение крепежа	0.32	0.18	0.23		
	Разрыв	0.33	0.49	0.39		
Faster R-CNN	Вмятина	0.08	0.17	0.11	обнаружение и классификация	0.25 сек/изобр
	Повреждение крепежа	0.68	0.12	0.20		
	Разрыв	0.12	0.42	0.18		
YOLO	Вмятина	0.98	0.92	0.95	обнаружение и классификация	0.05 сек/изобр
	Повреждение крепежа	0.94	0.78	0.86		
	Разрыв	0.93	0.77	0.86		

*время, которое требуется модели, чтобы сделать предсказание или выдать результат на входные данные.

Почему Faster R-CNN показала более низкие результаты?

- Faster R-CNN — двухэтапная архитектура, где сначала выделяются регионы интереса, а затем проводится классификация. При работе с небольшими объектами и слабовыраженными дефектами такая схема может терять точность локализации.
- Для корректной работы модели важны чётко выраженные объекты, тогда как исследуемые дефекты (вмятины, трещины, крепёжные повреждения) часто малозаметны, схожи по структуре и визуально не всегда сильно отличаются.
- Изменяющиеся условия съёмки, характерные для БПЛА (освещение, ракурс, фон), сильнее влияют на результаты Faster R-CNN, чем на более легковесные одноэтапные архитектуры.
- YOLOv11 в этой задаче оказалась более устойчивой и быстрой, обеспечив высокую точность детекции и классификации при низком времени инференса. Это и определило выбор YOLO в качестве основной модели.

Выводы по результатам сравнительного анализа

- По результатам сравнения трёх моделей (My_CNN, YOLOv11, Faster R-CNN) была выбрана YOLOv11 как наиболее сбалансированная по точности, скорости и устойчивости к внешним условиям.
- Модель обеспечивает высокие показатели F1-score по всем классам (до 0.95) и минимальное время инференса (~0.05 сек/кадр).
- В отличие от My_CNN, YOLO выполняет одновременную классификацию и локализацию, а в отличие от Faster R-CNN — работает быстрее и стабильнее на небольшом объеме данных.
- Применение YOLOv11 позволяет автоматизировать визуальный контроль состояния конструкции ЛА, делая возможным использование в реальных технических системах.

Сценарий применения и интеграции

По результатам исследования была выбрана модель **YOLOv11**, показавшая наилучшие характеристики по точности, скорости и устойчивости к условиям съёмки.

Предлагаемый сценарий использования включает:

- Съёмку внешней поверхности ЛА с помощью БПЛА
- Обработку отснятого видео (в офлайн-режиме) покадрово с помощью модели YOLOv11
- Выделение и классификация дефектов на каждом кадре
- Формирование отчёта в виде таблицы с указанием времени появления дефекта, его класса и confidence-оценки
- Предоставление результатов специалистам техобслуживания для последующего анализа

Также реализована возможность запуска модели в условно реальном времени — при подключении к камере или видеопотоку, результаты отображаются с минимальной задержкой, что позволяет использовать систему в интерактивном режиме мониторинга.

Таким образом, система может быть использована как в **постобработке видеоданных**, так и в **оперативной визуальной диагностике**, адаптируясь под конкретные условия эксплуатации.

Демонстрация работы модели YOLOv11 на видео



Заключение

- В рамках исследования разработана и протестирована нейросетевая система автоматического обнаружения и классификации дефектов на поверхности летательных аппаратов.
- Проведена подготовка обширного датасета, включающего три типа дефектов, с применением аугментации для повышения устойчивости моделей.
- Реализованы и сравнены модели: My_CNN, Faster R-CNN и YOLOv11.
- Модель YOLOv11 показала наилучшие результаты по точности, скорости обработки и устойчивости к внешним условиям.
- Разработан сценарий практического применения системы для диагностики с использованием видеосъёмки с БПЛА.
- Предложенный подход демонстрирует высокий потенциал для внедрения в системы технического осмотра и повышения безопасности авиационной техники.